REAL – TIME BIRD SOUND PREDICTION USING DEEP LEARNING AND IOT

A Project Report submitted in partial fulfilment of the requirements for the award of the degree of

MASTER OF SCIENCE IN DATA SCIENCE

Submitted by

Samuvel Jeba Kumar D

(2401722105024)

Under the guidance of

Mr. Doney I G Lawren



DEPARTMENT OF DATA SCIENCE
MADRAS CHRISTIAN COLLEGE (AUTONOMOUS)
CHENNAI – 59

APRIL – 2026

CERTIFICATE

This is to certify that the project work titled **"REAL – TIME BIRD SOUND PREDICTION USING DEEP LEARNING AND IOT"** submitted to the Madras Christian College, Chennai in partial fulfilment of the requirements for the award of the degree, **MASTER OF SCIENCE IN DATA SCIENCE** is a record of original project work done by **SAMUVEL JEBA KUMAR D (2401722105024)** during the period **2025-2026** of his study in the Department of Data Science, Madras Christian College, Chennai under the supervision and guidance of **Mr DONEY I G LAWREN**, B.Sc., MCA., Assistant Professor, Department of Data Science, Madras Christian College, Chennai and the project has not formed the basis for the award of any Degree/ Diploma / Associateship/ Fellowship or other similar title of any candidate of any University.

Dr. E. MAHIMA JANE
Assistant Professor In-Charge
Department of Data Science
Madras Christian College

Mr. DONEY I G LAWREN
Project Guide, Assistant professor
Department of Data Science
Madras Christian College

Submitted for the Via-Voce Examination held on_____

Internal Examiner **External Examiner**

DECLARATION

I, **SAMUVEL JEBA KUMAR D**, hereby declare that the project entitled **"REAL – TIME BIRD SOUND PREDICTION USING DEEP LEARNING AND IOT"** submitted to the Madras Christian College, Chennai in partial fulfilment of the requirements for the award for the Degree of **M.Sc. DATA SCIENCE** is a record of original and independent project work done by me during **2025 – 2026** under the Supervision and Guidance of **Mr DONEY I G LAWREN**, B.Sc., MCA., Assistant Professor, Department of Data Science, Madras Christian College, Chennai and it has not formed the basis for the award of any Degree / Diploma / Associateship / Fellowship or other similar title to any candidate in any University.

Date: **Signature of the candidate**

ACKNOWLEDGEMENT

This is my Final Year Project report where I have a lot of helping hands who deserve the warmest thanks.

First, I would like to express my gratitude and love to God for providing me a wonderful life and blessing me with a beautiful family.

I thank **Dr. P Wilson**, The Principal & Secretary for providing me the opportunity to pursue in this prestigious institution and gain work space experience for the future purposes.

I thank **Dr. E. Mahima Jane**, The Head of the Department of Data Science, for her words of wisdom and motivation.

My sincere thanks to my internal project guide, **Mr. Doney I G Lawren,** Assistant Professor, Department of Data Science, who deserves a lot of credit for guiding me during the project period with his constant support and his words of encouragement.

I am grateful to **Dr. C. Joyce Priyakumari,** The Head of the Department of Zoology and the Department of Zoology for giving me this wonderful opportunity for me to gain knowledge and to get a work space at their esteemed department.

I would like to express my heartfelt Gratitude to External Guide **Dr. A. Anita Pearline Esther Assistant Professor, Department of Zoology** who spent her valuable time despite his busy schedule in teaching new Methods, techniques and analysis in the field of Data Science.

Overall, this 4 month of internship was a fruitful one by grasping a lot of information, improving my skills, and understanding concepts under a working atmosphere.

Samuvel Jeba Kumar D

ABSTRACT

Monitoring bird species is an important part of studying the environment and understanding biodiversity. At Madras Christian College, many different types of birds live in various areas of the campus. Some of these birds stay on the campus all year, while other birds migrate to different places depending on the season. Because of this, trying to manually identify and monitor which bird is present in which location every single day is a very difficult task. Traditional manual methods take up a lot of time and require human effort. Therefore, continuous and accurate monitoring is challenging without an automated system, and an automated system is heavily needed to solve this problem.

To address these challenges, a real-time bird sound prediction system was developed building a real-time bird sound prediction system using deep learning and Internet of Things (IoT) technologies. The hardware setup is designed to be low-cost, efficient, and capable of real-time monitoring. A microphone is utilized to capture live bird vocalizations from the campus environment. For the main processing unit, an Arduino Nano 33 BLE was selected as the primary processing unit. The microphone connects directly to this Arduino, and the Arduino processes the audio data right there on the device. Because the Arduino Nano 33 BLE handles the heavy machine learning part, it was interfaced with an ESP8266 Wi-Fi module using serial communication (RX and TX pins) to handle the internet connection.

Before the hardware could predict anything, we had to collect data and extract MFC audio features to train our deep learning models. A dataset was developed by aggregating manually recorded sounds, audio files from the Merlin Bird App, and various species recordings from the Xeno-Canto online dataset. During the software development phase, we explored and trained three different deep learning models using Google Colab: CNN, LSTM, and Transformer models. We compared how they worked for classifying bird audio. However, for the final hardware deployment, we chose to deploy only the CNN model on to the Arduino Nano 33 BLE for the live prediction tasks.

The final working process of the project happens entirely in real time. When a bird makes a sound, the microphone captures the audio. The Arduino Nano 33 BLE immediately runs the deployed CNN model to process the audio features and instantly identify the bird species. After the bird is identified, the Arduino sends the predicted bird name as text to the ESP8266 module. The ESP8266 acts as a bridge and immediately sends this text result over

Wi-Fi to a Supabase cloud database. Finally, these bird detection results can be easily viewed on a mobile phone or a computer screen. This complete process creates a very helpful system for continuous campus bird monitoring.

Keywords: Bird Sound Prediction, Deep Learning, IoT, Arduino Nano 33 BLE, ESP8266, Supabase, CNN, Real-Time Monitoring, Xeno-Canto.

INDEX

CHAPTER	TABLE OF CONTENTS	PAGE NO:
1	**INTRODUCTION**	1
	1.1 Overview	1
	1.2 Types of Birds and Monitoring Methods	1
	1.3 Problem Statement	2
	1.4 Project Usage and Significance	3
	1.5 Procedure and System Creation	3
	1.6 Hardware Selection and Device Roles	3
	1.6.1 Device Specifications: Arduino Nano 33 BLE	3
	1.6.2 Device Specifications: ESP8266 Wi-Fi Module	5
	1.6.3 Hardware Connections, Pinouts, and Power Supply	6
	1.6.4 Cloud Architecture: Supabase Platform	6
	1.7 Objectives of the Research	7
	1.8 Organization of the Chapters	8
2	**REVIEW OF LITERATURE**	9
	2.1 Introduction	9
	2.2 Deep Learning for Bird Sound Classification	9
	2.3 Hardware Components Research	10
	2.3.1 Arduino Nano 33 BLE	10
	2.3.2 ESP8266 Wi-Fi Module	12
3	**METHODOLOGY AND SYSTEM DESIGN**	13
	3.1 Introduction	13
	3.2 Methodology Workflow	13
	3.2.1 Requirement Analysis	13
	3.2.2 Data Collection	13
	3.2.3 Audio Pre-processing	14
	3.2.4 Model Selection and Training	14
	3.2.5 IoT Hardware Integration	14
	3.2.6 Real-Time Edge Prediction	14
	3.2.7 Cloud Communication and Database Storage	15

	3.3 System Architecture Overview	15
	3.4 System Environment	15
	3.5 Data Collection and Pre-processing	16
	3.5.1 Gathering Acoustic Data	16
	3.5.1.1 Merlin Bird App	17
	3.5.1.2 Xeno-Canto database	18
	3.5.1.3 Manually recorded	18
	3.5.2 Audio Standardization	19
	3.5.3 Pre-processing using the MFC Model	19
	3.6 Model Selection and Training Phase	20
	3.6.1 Exploring Deep Learning Architectures	20
	3.6.2 The Rationale for Selecting CNN	20
	3.6.3 Training and Conversion for Edge	21
	3.7 Hardware Setup and Edge Deployment	21
	3.7.1 The Microcontroller: Arduino Nano 33 BLE	21
	3.7.2 Audio Interfacing and Real-Time Inference	22
	3.7.3 IoT Integration and Cloud Storage	22
	3.7.4 The Communication Bridge: ESP8266	22
	3.7.5 Cloud Database: Supabase	24
	3.8 Overview Project	24
4	**HARDWARE AND SOFTWARE IMPLEMENTATION**	26
	4.1 Introduction	26
	4.2 Hardware Components Used	26
	4.2.1 Arduino Nano 33 BLE (The Processing Core)	26
	4.2.2 ESP8266 Wi-Fi Module (The Network Messenger)	27
	4.2.3 Analog Microphone Module (The Acoustic Sensor)	27
	4.3 Circuit Design and Wiring	27
	4.4 Software Environment and Tools	29
	4.5 Implementation of the Workflow	29
	4.6 Detailed Code Logic	31
	4.6.1 Deep Learning Model & Training (TensorFlow)	31

	4.6.2 Real-Time Edge Inference (Arduino Nano 33 BLE)	31
	4.6.3 Data Transmission & Cloud Integration (ESP8266 & Supabase)	32
	4.7 Supabase Database Setup	32
5	**RESULTS AND DISCUSSION**	34
	5.1 Introduction	34
	5.2 Model Training and Software Results	34
	5.3 Hardware and Real-Time Testing	38
	5.4 Cloud Integration and Network Results	39
	5.5 Real-Time System Operation and Live Results	39
	5.5.1 Live Edge Prediction (Arduino Nano 33 BLE)	39
	5.5.2 Hardware Serial Routing (ESP8266 Module)	40
	5.5.3 Remote Database Logging (Supabase Platform)	41
	5.6 Discussion and System Limitations	42
6	**CONCLUSION**	43
7	**FUTURE ENHANCEMENTS**	44
8	**REFERENCES**	46
	APPENDIX	48

LIST OF ABBREVIATIONS

S.NO	ABBREVIATION	EXPANSIONS / MEANING
1	ADC	Analog-to-Digital Converter
2	BLE	Bluetooth Low Energy
3	FPU	Floating-Point Unit
4	GND	Ground (electrical connection)
5	IMU	Inertial Measurement Unit
6	IoT	Internet of Things
7	RX	Receive (serial data line)
8	TX	Transmit (serial data line)
9	UART	Universal Asynchronous Receiver-Transmitter (serial communication)
10	USB	Universal Serial Bus
11	API	Application Programming Interface
12	HTTP	Hypertext Transfer Protocol
13	IDE	Integrated Development Environment
14	JSON	JavaScript Object Notation
15	MQTT	Message Queuing Telemetry Transport
16	RAM	Random Access Memory
17	SSID	Service Set Identifier (Wi-Fi network name)
18	TCP/IP	Transmission Control Protocol/Internet Protocol
19	CNN	Convolutional Neural Network
20	CRNN	Convolutional Recurrent Neural Network
21	LSTM	Long Short-Term Memory
22	MFC	Mel-Frequency Cepstrum
23	MFCC	Mel-Frequency Cepstral Coefficients
24	TinyML	Tiny Machine Learning

LIST OF FIGURES

S.NO	FIGURE NO	TITLE	PAGE NO:
1	1.1	Arduino Nano 33 BLE Overview	4
2	1.2	ESP8266 Wi-Fi Module Overview	5
3	3.1	Merlin Bird App Data Set	17
4	3.2	Screen-Recorded to MP3 Audio	17
5	3.3	Xeno-Canto Database	18
6	3.4	Manually Recorded Dataset	18
7	3.5	MP3 and WAV File Samples	19
8	3.6	Arduino Nano 33 BLE	22
9	3.7	ESP8266 Module	23
10	3.8	Jumper Wires	23
11	3.9	Overview Project	25
12	4.1	Circuit Design and Wiring	28
13	4.2	Implementation of the Workflow	30
14	4.3	Supabase Database Setup	33
15	5.2	Accuracy Chart	36
16	5.3	Confusion Matrix	36
17	5.4	Training Accuracy Curves	37
18	5.5	Training Loss and Accuracy Curves	38
19	5.6	Live Edge Prediction (Arduino Nano 33 BLE)	40
20	5.7	Hardware Serial Routing (ESP8266 Module)	41
21	5.8	Remote Database Logging (Supabase Platform)	41

LIST OF TABLES

S.NO	TABLE NO	TITLE	PAGE NO:
1	5.1	Model Accuracy Comparison Table	35

1. INTRODUCTION

The study of bird populations is a great way to understand the health of our environment. Because birds are very sensitive to changes in nature, monitoring them gives us important clues about the local ecology. This project focuses on the Madras Christian College campus, which is a beautiful green area that attracts many different kinds of birds. Our goal is to create an automated way to track these creatures to eliminate the dependency on constant manual labor.

1.1 Overview

Watching birds is one of the best ways to check if our environment is doing well. Because birds perceive environmental fluctuations very quickly, their habits and sounds tell us a lot about how healthy the local area is. Madras Christian College is distinguished by its lush biodiversity that stays full of different types of birds. Keeping a simple and steady record of these birds is important for students and teachers who want to protect the nature around the college.

However, the traditional manual observation methods have some big problems. Right now, someone has to walk around the thick trees and try to spot birds with their own eyes, which is exhausting and takes a lot of time. It is also impossible for a person to be everywhere at once or to watch the birds all night long. Because it is so hard to keep track of every bird manually, we really need a smart system that can do this work on its own without requiring manual supervision all day.

This project solves that problem by building a smart device designed for acoustic recognition that performs automated species classification. We used an Arduino-based microcontroller to listen to the sounds, and a Wi-Fi chip to transmit classification data directly to a digital logbook on the internet. This means the device can sit outside in the trees and work 24/7 without getting tired. Instead of someone having to do the hard labor of manual bird watching, our system creates an easy-to-read digital list that shows exactly which birds were on campus and when they showed up.

1.2 Types of Birds and Monitoring Methods

Madras Christian College is a very large place with many different spots where birds like to stay. On our campus, we have two main groups of birds: those that live here all the time

and those that fly in from other places when the weather changes. These birds are very smart and sensitive; if they hear loud human noises or feel disappointed by too much activity, they may fly away, remain silent, or hide deep inside the trees. Because they move from one part of the campus to another to find peace, it is very hard for students to keep track of them just by walking around.

Usually, students try to watch these birds with their eyes, but it is a big challenge because the birds are fast and the leaves are very thick. A much better way to find them is by listening. Even if a bird is sitting perfectly still and "idle" in a tree where you cannot see it, its vocalizations indicate their precise location where it is. Since birds utilize acoustic communication to each other all the time, listening to them is the most reliable way to know they are there.

To facilitate research for students and researchers, we created this device to act as an automatic listener. Instead of a student having to stand in the heat or noise waiting for a bird to appear, this system stays out in the different campus locations and automates the data collection process. It predicts the bird sounds, and the information is digitized and can be saved for the students to study later.

1.3 Problem Statement

Right now, Madras Christian College has a wide variety of birds spread across the campus, but keeping track of them manually every day is significantly challenging. The usual way of doing this involves a person walking around, waiting quietly, and trying to spot or hear the birds themselves.

This manual way of doing things has big problems. It takes way too much time and human energy. Someone cannot be everywhere at once. Also, since some birds stay put while others migrate, keeping an accurate, non-stop record is basically impossible without a machine helping out. Due to these challenges, an automated monitoring system is critically required. to take over the manual work.

1.4 Project Usage and Significance

This project provides a significant asset for biological and environmental research at the college. It creates an automatic tool that works around the clock without getting tired. We can deploy the system in specific spots around the campus to automatically record exactly when a certain bird shows up. Students and teachers can then use this data to study bird habits, track migrations, and monitor the overall health of the campus ecosystem for years to come. It replaces the labor-intensive process of bird watching with a structured digital database

1.5 Procedure and System Creation

To solve the issues with manual bird watching, we built an automated system that identifies bird calls using deep learning and IoT technology. The project follows a specific workflow to ensure speed and accuracy. First, we use a high-sensitivity microphone to capture live audio from the environment. This sound is immediately processed by a CNN model running directly on the Arduino, allowing the system to "predict" the bird species in real-time without needing a heavy computer.

Once a bird is identified, the data is handed over to an ESP8266 Wi-Fi module. This module acts as a bridge, sending the species name and time of day to a Supabase cloud database. By processing the audio at the edge (locally on the hardware) before sending a tiny text alert to the web, the system stays fast and uses very little internet data. This creates a live, digital record of campus wildlife that works 24/7 without any human help.

1.6 Hardware Selection and Device Roles

To actually make edge computing work, we couldn't just use any basic electronics. We had to carefully split the workload across two very specific hardware devices so the system wouldn't crash.

1.6.1 Device Specifications: Arduino Nano 33 BLE

The primary processing unit selected for the core computing tasks is the Arduino Nano 33 BLE. As shown in Figure 1.1,In traditional electronics projects, older boards like the Arduino Uno are commonly used, but those 8-bit microcontrollers lack the computational capacity required for modern artificial intelligence tasks. The Nano 33 BLE was specifically

designed to handle embedded machine learning, making it the primary processing unit for the acoustic monitoring system.

Architecturally, this board features a 32-bit Arm Cortex-M4F processor running at a clock speed of 64 MHz. It also contains 1 MB of flash memory and 256 KB of RAM. This extra memory is the exact reason this specific device was chosen. Deep learning models, even compressed ones, require a significant amount of temporary memory to process audio arrays. Standard boards would instantly crash when trying to load a Convolutional Neural Network, but the Nano 33 BLE has enough space to hold the model and perform the mathematical operations without lagging.

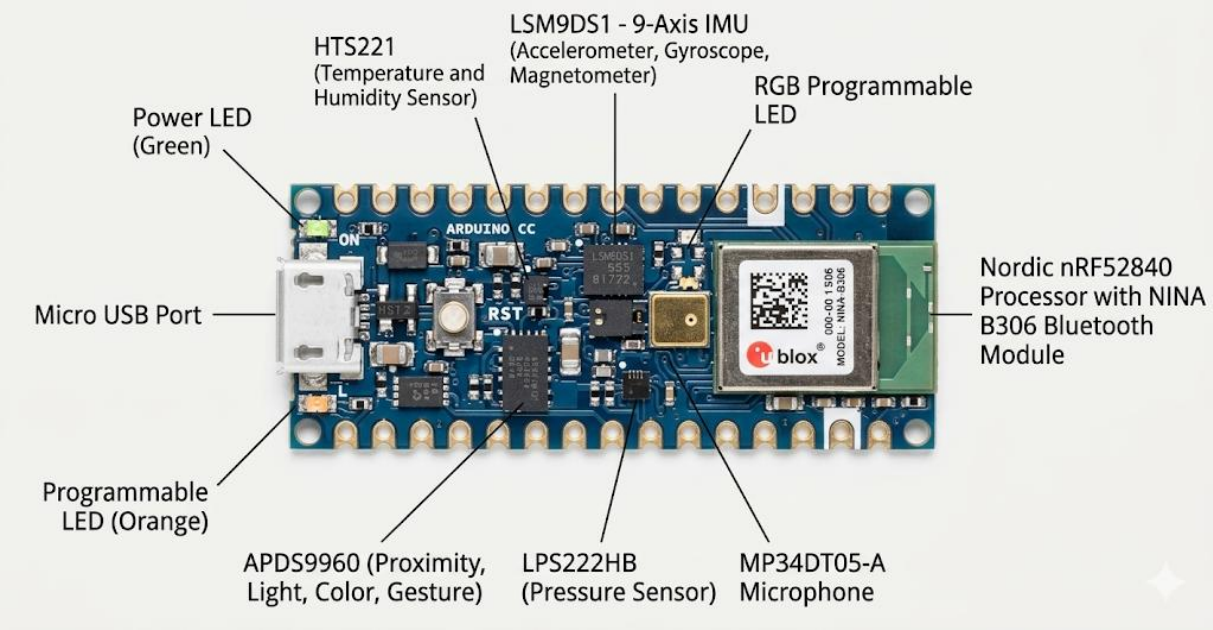


Figure 1.1 Arduino Nano 33 BLE Overview

The procedure for using this device begins with the software setup. The compressed TensorFlow Lite model is deployed directly to the microcontroller's flash memory. During active operation, the Arduino runs in an infinite loop. It constantly reads analogue voltage signals from the attached microphone, essentially "listening" to the environment. The device captures a window of audio, processes the sound wave into mathematical features, and pushes those numbers through the neural network. If the network identifies a bird call, the Arduino generates a simple text string containing the species name. Because this board does not have native Wi-Fi capabilities, it must immediately push this text data out through its serial transmission pins to a secondary device.

1.6.2 Device Specifications: ESP8266 Wi-Fi Module

The ESP8266 is a highly popular, low-cost Wi-Fi microchip with a full built-in TCP/IP networking stack. As shown in Figure 1.1, the Arduino handles the complex artificial intelligence math. As shown in Figure 1.2, the ESP8266 is responsible entirely for internet connectivity.

The decision to use a completely separate device for network tasks was made to ensure system stability. Managing a Wi-Fi connection requires a microcontroller to constantly check for network dropouts, handle security handshakes, and format internet protocols. If the Arduino was forced to handle these network background tasks while simultaneously trying to listen to audio and calculate neural network predictions, the system would experience severe bottlenecking and eventually freeze. By offloading all internet duties to the ESP8266, the hardware workload is safely divided.

The working procedure for the ESP8266 is very straightforward. Upon receiving power, the device runs a setup script that commands it to search for the local campus Wi-Fi router. Using pre-programmed credentials (an SSID and password), it logs into the network and establishes a stable internet connection. Once connected, the device enters a waiting state. It continuously monitors its hardware receiving pin, listening for any incoming messages from the Arduino. When a text string arrives for example, the words "Asian Koel" the ESP8266 wakes up. It packages that text into a structured JSON format, opens a connection to the internet database, and fires off an HTTP POST request. After the transmission is successful, the module returns to its silent waiting state.



Figure 1.2 ESP8266 Wi-Fi Module Overview

1.6.3 Hardware Connections, Pinouts, and Power Supply

Properly wiring the electronic components is a critical phase of the physical implementation. Because the system relies on two separate microcontrollers communicating with each other, the circuit design must be handled carefully to prevent electrical shorts or data corruption. The system is powered using a standard USB cable connected to a power bank or wall adapter. The USB cable plugs directly into the micro-USB port of the Arduino Nano 33 BLE, supplying a standard 5 volts of power. However, the internal logic of both the Arduino and the ESP8266 operates on 3.3 volts. The Arduino has a built-in voltage regulator that steps the 5V USB power down to a safe 3.3V. The power pins on the ESP8266 and the external microphone are then connected to the 3.3V output pin on the Arduino, ensuring that all components receive a safe and equal power supply without needing multiple batteries.

Data sharing between the two boards is handled through UART hardware serial communication. The procedure for connecting serial devices requires crossing the data lines. The TX (Transmit) pin on the Arduino must be connected to the RX (Receive) pin on the ESP8266 using a jumper wire. Conversely, the RX pin on the Arduino is connected to the TX pin on the ESP8266. This cross-wiring allows the text data to flow smoothly from the sender to the receiver.

The most important step in the entire wiring process is the ground connection. A jumper wire must be connected from the GND (Ground) pin of the Arduino directly to the GND pin of the ESP8266. If a common ground is not established, the electrical voltage references will not match between the two boards. Without a shared ground, the text data sent across the TX/RX wires will become corrupted, and the ESP8266 will only read unreadable garbage symbols.

1.6.4 Cloud Architecture: Supabase Platform

After the hardware successfully processes the audio and transmits the data over Wi-Fi, the final destination for that information is the cloud. Instead of writing a complex web server from scratch or purchasing expensive hosting plans, the system utilizes Supabase.

Supabase is a modern, open-source backend-as-a-service platform. It essentially provides developers with a ready-to-use PostgreSQL database that is permanently connected

to the internet. PostgreSQL is a highly structured database system that organizes information into neat rows and columns, making it the perfect environment for logging long lists of sensor data. The procedure for using Supabase starts with configuring the project dashboard. A specific table named bird detections was created within the database. This table was designed with strict columns: an auto-incrementing ID number, a text column for the bird's name, and a timestamp column.

When the ESP8266 device sends its HTTP POST request over the campus Wi-Fi, the data packet travels directly to the Supabase API endpoint. The platform uses a unique project API key to securely authenticate the hardware device. Once the security check passes, Supabase extracts the plain text bird name from the JSON packet and inserts it into a new row in the database. A major advantage of this platform is its automated timestamping feature. As soon as the new row is created, the Supabase server automatically generates the exact date and time and locks it into the database. This completely removes the need for the physical hardware to keep track of a real-time clock, saving memory and keeping the code highly efficient. The final result is a clean, live-updating digital logbook that can be viewed from any web browser.

1.7 Objectives of the Research

The primary goal of this study is to create a fully automated system that tracks birds across the Madras Christian College campus without needing any human help. To achieve this, the project focuses on the following specific points.

- To design and develop a deep learning model for bird sound classification.
- To apply deep learning techniques for accurate bird sound prediction.
- To implement a IoT-based system for real-time audio capture and analysis.
- To deploy the trained model on a device for live prediction.
- To send bird detection results to a mobile or computer through supabase.
- To build a low-cost, efficient, and real-time bird monitoring system.

Our setup has some major benefits over the old ways of tracking birds. By using microphones instead of cameras, the system works just as well at night as it does during the day. It also doesn't care if leaves or branches are in the way.

On top of that, running the audio processing locally on the Arduino Nano 33 BLE makes the whole thing very efficient. If we had to stream live audio to the cloud constantly, it would waste tons of internet data and battery power. By figuring out the bird's name right on the device and only sending a tiny text message through the ESP8266 to Supabase, the system is fast and uses barely any Wi-Fi. This keeps costs down and makes it perfect for leaving outside around the campus for a long time.

1.8 Organization of the Chapters

To help the reader navigate the report, the documentation is divided into the following chapters:

- **Chapter 2: Literature Review** – This section explores past studies and existing technologies used for identifying bird sounds and working with small microcontrollers.
- **Chapter 3: Methodology and System Design** – This chapter describes the project's workflow, including how we gathered audio samples and designed the deep learning model.
- **Chapter 4: Hardware and Software Implementation** – Here, we detail the physical construction of the device, covering the wiring of the sensors and the connection to the cloud database.
- **Chapter 5: Results and Discussion** – This part presents the final data, showing how well the model performed during live tests on the campus.
- **Chapter 6: Conclusion** – The final summary of the project's success in automating bird tracking and replacing manual field labor.
- **Chapter 7: Future Scope and Enhancements** – This chapter offers a look at how the system can be improved with more powerful hardware and solar power for global use.

2. REVIEW OF LITERATURE

2.1 Introduction

This section explores existing research on deep learning for bird sound classification. It examines various neural network architectures and the hardware capabilities of microcontrollers used for edge computing.

2.2 Deep Learning for Bird Sound Classification

- **Shubham Revadekar, Vidhish Panchal, Pratik Kanani, Kamal Shah, Anil Vasoya, Ravikumar Pandey** (2023) focuses on bird sound classification using deep learning. The research utilizes the BirdCLEF 2022 dataset from Xeno-Canto, converting audio signals into Mel spectrograms with MFCC features to enable image-based CNN analysis. The authors compare several transfer learning models, including Xception, InceptionV3, ResNet50, EfficientNet, and VGG16, alongside two custom CNN architectures. Results indicate that deep learning achieves high accuracy, with Xception and the custom CNN performing best. The study demonstrates that spectrogram-based analysis and transfer learning enhance bird species identification monitoring.

- **Deepak T. Mane, Pravin Kumar Bhoyar, Govinda B. Sambare, Gitanjali S. Mate, Gitanjali S. Mate, Archana Jadhav, Dipali Himmatrao Patil.** (2026) presented a specialized Convolutional Neural Network (CNN) for bird species identification. By leveraging Mel frequency cepstral coefficients (MFCCs), their model achieved 93.4% accuracy, significantly enhancing real-time ecological monitoring and biodiversity conservation efforts.

- **Indumathi C P, Diviya lakshmi K R and Mahalakshmi R** (2024) utilized a CNN-LSTM architecture to process bird vocalizations from the Xeno-canto and Cornell databases. By extracting spatial features via CNNs and temporal dependencies through LSTMs, the system effectively analyzed preprocessed spectrograms and MFCCs. The model achieved 75% accuracy, demonstrating the effectiveness of combining spatial and temporal feature extraction for ecological acoustic monitoring.

- **Ashish Kumar Pandey, Sanyam Kudale, Rohit Jadhav, Swaraj Giri, S. K. Wagh** (2025), proposed an automated system for bird species identification utilizing deep learning techniques on vocalization data. By employing a hybrid CRNN architecture trained on processed Xeno-Canto recordings incorporating features such as MFCC and spectrograms. The study reported a classification accuracy of 97%. However, the authors noted persistent challenges regarding environmental noise interference and the computational feasibility of real-time deployment.

- **Sinduran Sivarajan** (2016) evaluated machine learning techniques for bird sound classification using the RefSys Animal Sound Archive comparing traditional classifiers like SVM and KNN against deep learning models, the study utilizes both engineered MFCC and automatic feature extraction. The findings highlight Artificial Neural Networks as the top performer, achieving a 91% accuracy rate and demonstrating the efficacy of neural architectures in bioacoustic analysis.

2.3 Hardware Components Research

2.3.1 Arduino Nano 33 BLE

- Another highly relevant study for our system design is the work by **Kurniawan** (2021) in his guide, IoT Projects with Arduino Nano 33 BLE Sense. In this book, he explores the practical applications of the exact microcontroller family selected for our project. Kurniawan explains that this board is driven by an nRF52840 chip and features Bluetooth Low Energy (BLE), making it very effective for setting up short-range wireless communication networks. He also provides a detailed look at the board's impressive array of built-in sensors such as those for tracking temperature, pressure, and movement which communicate easily through standard I2C and SPI connections.

  Kurniawan successfully showed how to use TensorFlow Lite to run machine learning models directly on the Arduino. In his examples, he used the board's processing power to classify physical gestures locally, entirely without the need for an internet connection or cloud processing. Along with providing clear instructions for setting up the Arduino IDE software on different operating systems, his work ultimately confirms that the Arduino Nano 33 BLE is a cheap, powerful, and highly capable platform for building smart edge-computing devices.

- A 2025 technical evaluation by **Jeenali Shah, Rutuja Phalle, Vaishnavi Renke, and Dr. Pravin Gawande** provided an in-depth hardware teardown of the Arduino Nano 33 BLE Sense. The research isolated the exact reasons this microcontroller succeeds in localized edge computing, placing a heavy focus on the underlying Nordic ARM Cortex-M4 processing chip. The research team thoroughly evaluated the physical board, running specific stress tests on the built-in audio, motion, and environmental sensors. A massive part of the study revolved around forcing artificial intelligence algorithms to run directly on the bare metal. To accomplish this, the group paired the TinyML software framework with the native Bluetooth Low Energy communication stack. By intentionally maxing out the physical memory constraints and tracking the data transfer rules, the findings confirmed that the small module remains completely stable for standalone smart devices. The final takeaway noted that the minimal electricity required to keep the board running makes it a perfect baseline for continuous observation tools, pointing specifically to wearable medical monitors, factory network trackers, and agricultural field sensors.

- **Viswanatha V.** and a research team published a practical study in 2022 demonstrating exactly how to run Tiny Machine Learning (TinyML) directly on edge computing hardware. To perform their physical tests, the group utilized the Arduino Nano 33 BLE Sense microcontroller. They specifically tapped into the board's onboard microphone alongside its built-in six-axis inertial measurement unit (IMU). By utilizing these specific sensors, the team forced the small hardware module to execute two completely separate jobs. One part of the system tracked physical hand gestures moving through the air, while the other part actively listened for human speech. To make the microcontroller smart enough to handle this, the researchers built and compressed custom neural networks using the Edge Impulse software platform. After the training phase, the models were integrated into the Arduino microcontroller. The hardware successfully classified the physical hand movements and caught specific vocal commands without ever touching a Wi-Fi connection. To prove the speech recognition worked, the setup was programmed to listen for certain spoken keywords and instantly flash the board's RGB LEDs in matching colors. The final outcome of this research proved that heavy artificial intelligence algorithms can actually be squeezed onto tiny,

low-power chips. Doing this allows engineers to build highly secure and instantly responsive human-to-computer interfaces that never have to rely on cloud processing.

2.3.2 ESP8266 Wi-Fi Module

- In their 2021 study, **Visan and Diaconu** basically proved that you don't need to drop a ton of cash to have a smart home. They built a DIY system using an **ESP8266** chip and the Blynk app, which is way cheaper than the commercial solutions you'd find at a store. The setup is actually pretty straightforward. They used a DHT22 to track temperature/humidity and a BH1750 sensor to see how bright the room is. All that data just gets sent right to your phone via Wi-Fi. But the real "win" is the four-channel **relay** they included. It's what lets you actually flip your lights or fans on and off remotely, basically turning any old Android or iPhone into a DIY remote control. The authors really wanted to show that while most people think smart homes are too expensive, you can actually get it done with basic, budget-friendly hardware if you're willing to build it yourself.

- **Likhitha U N, Manjunath R, and Darshan N** published an applied research paper in 2024 detailing the construction of a low-cost Internet of Things (IoT) home automation network. The physical build for their project relies heavily on the ESP8266 microcontroller. This specific chip was selected to act as the main hub because its native Wi-Fi capabilities allow standard household electronics like ceiling fans, lightbulbs, and air conditioning units to link directly to a local router. To safely handle the electrical current required to physically turn these heavy appliances on and off, the researchers wired the microcontroller straight into a standard multi-channel 5V relay module. For the software architecture, the system transmits data using the lightweight MQTT messaging protocol. This specific protocol acts as a reliable bridge between the physical relay hardware and a custom-built web dashboard. By structuring the network this way, an individual can simply open a smartphone from any remote location to monitor their living space and toggle power to their electronics. The final conclusion of their work proved that building such an accessible framework drastically improves daily household convenience while simultaneously cutting down on wasted electricity.

3. METHODOLOGY AND SYSTEM DESIGN

3.1 Introduction

Manual observation of bird species at Madras Christian College is restricted by the campus's dense vegetation. Visual tracking requires excessive human labor and often yields incomplete data. The objective of this project was to engineer an autonomous acoustic sensor capable of identifying bird calls locally and logging the results to a remote database.

The physical constraints of an outdoor deployment dictated the entire project pipeline. Because the hardware had to operate independently, the workload was divided into two distinct engineering pipelines. The software phase focused on gathering field audio, processing the acoustic arrays using the Mel-Frequency Cepstrum (MFC) model, and training an image-based neural network. The hardware phase required embedding that trained model onto an edge-computing microcontroller, capturing live audio buffers, and transmitting lightweight text payloads over a wireless network.

3.2 Methodology Workflow

3.2.1 Requirement Analysis

Before touching any hardware, the first step was figuring out exactly what physical constraints we were working with for remote outdoor listening. The setup needed a small processor that could handle computationally intensive tasks on the edge. It also needed a separate, stable network gateway to push data to the internet. Because the entire rig had to run on a standard portable power bank, keeping power consumption as low as possible dictated our choice of low-voltage microcontrollers.

3.2.2 Data Collection

To actually train the prediction model, a highly specific library of audio samples had to be built. Sound clips of target local species were gathered and isolated of local bird species, specifically targeting the Asian Koel. Once collected, these raw audio files were chopped into equal-length segments. This editing process cut out dead air and background noise, forcing the neural network to focus only on the actual bird vocalizations rather than the environment.

3.2.3 Audio Pre-processing

Feeding raw audio waves straight into a basic microcontroller is mathematically too heavy. To get around this, the chopped audio segments were mathematically converted using a Mel-Frequency Cepstrum (MFC) calculation. This math essentially transforms an audio signal into a static visual spectrogram. By doing this, the system pulls out the unique distinctive acoustic signature of a bird's call, setting up the data perfectly for an image-based neural network.

3.2.4 Model Selection and Training

Selecting a Convolutional Neural Network (CNN) was a deliberate choice aimed at reaching the highest possible accuracy for this project. During the initial testing phase, The CNN model was compared against other architectures like LSTMs and Transformers to see which would work best. The results showed that the CNN consistently provided the most precise results when analyzing the complex patterns in bird sounds. While other models suffered from unstable performance or struggled with background noise, the CNN proved to be much more reliable in the actual campus environment. Once the high-accuracy model was finalized, I optimized it to fit within the limited memory of the Arduino hardware. This allowed the system to maintain its full predictive power during live field deployment without crashing or lagging.

3.2.5 IoT Hardware Integration

The physical build was put together by linking two different microcontrollers on a single breadboard. For the offline audio processing, an Arduino Nano 33 BLE Sense was used since it features a built-in microphone. To give the system internet access, an ESP8266 Wi-Fi chip was added to the circuit. The two boards were hooked up using a hardware UART serial line, crossing the transmit (TX) and receive (RX) wires so they could safely pass text messages back and forth.

3.2.6 Real-Time Edge Prediction

When powered on outdoors, the Arduino runs an endless listening loop. It grabs quick snapshots of audio from the mic, runs the MFC feature extraction locally, and pushes that math through the compressed CNN model. If the network identifies a recurring pattern it recognizes,

the Arduino just grabs that specific bird's name and fires it out as a plain text string over its transmit pin.

3.2.7 Cloud Communication and Database Storage

The very last part of the system handles the automated logging. While the Arduino does the heavy predictive lifting, the ESP8266 simply waits and monitors its receive pin. The second a bird name comes down the wire, the Wi-Fi module grabs the text, wraps it into a standard JSON format, and builds an HTTP POST request. It then shoots that request over the campus Wi-Fi straight to a remote Supabase database, permanently logging the species name and the exact timestamp into an online table.

3.3 System Architecture Overview

Transmitting raw acoustic data from a remote sensor to a cloud server is impractical. Continuous audio streaming consumes massive amounts of power and immediately fails if the local Wi-Fi connection drops. The architecture for this project was built entirely around edge computing to bypass these network limitations.

The sensing node relies on an Arduino Nano 33 BLE to handle the heavy computational workload. A microphone captures ambient sound, and the Arduino processes the audio through an embedded Convolutional Neural Network (CNN). The hardware identifies the bird species locally and then clears the raw audio from its memory buffer.

Instead of an audio file, the output becomes a simple text string containing the species name. The Arduino passes this text data over a serial hardware connection to an ESP8266 Wi-Fi module. The ESP8266 acts strictly as an internet bridge, taking the serial string and pushing it to a Supabase database. Processing the data at the edge reduces the transmission payload from megabytes of audio to a few bytes of text.

3.4 System Environment

To actually write the code, train the models, and program the boards, we had to use a specific mix of software tools. Training deep learning models on standard consumer hardware is computationally prohibitive and highly inefficient. To circumvent hardware limitations, machine learning computations were executed within cloud environments using Google Colab

and Edge Impulse. This let us borrow their remote GPUs for free to speed up the training process.

For the actual coding, we wrote the neural network scripts in Python. We relied heavily on the standard TensorFlow and Keras libraries to build and test the CNN layers. After the model hit our accuracy goals, we used the TensorFlow Lite for Microcontrollers framework. This specific tool was a lifesaver because it took our massive Python model and compressed it down into a tiny C++ header file.

On the hardware side, everything was programmed using the classic Arduino IDE. We wrote the physical device code in C++. The Arduino IDE was used to compile the code, link the TensorFlow Lite libraries, and flash the final scripts onto both the Arduino Nano 33 BLE and the ESP8266 over a USB connection.

3.5 Data Collection and Pre-processing

The classification model required a highly specific dataset tailored to the actual deployment environment. Pre-packaged acoustic datasets lack the specific background noise profile of the college campus, which would cause the neural network to fail during field deployment.

3.5.1 Gathering Acoustic Data

We compiled the training data from three separate sources. Local field recordings were captured manually across the campus grounds. This allowed the model to map persistent background frequencies, including wind across the microphone, insect activity, and distant student traffic.

Verified acoustic samples were pulled from the Merlin Bird App to establish clean, high-fidelity baselines for the target species. To meet the high-volume requirements of deep learning, bulk audio files were downloaded from the Xeno-Canto database. This external repository provided hundreds of variations of specific bird calls, warning signals, and mating songs, ensuring the model did not overfit to a single clean recording.

3.5.1.1 Merlin Bird App

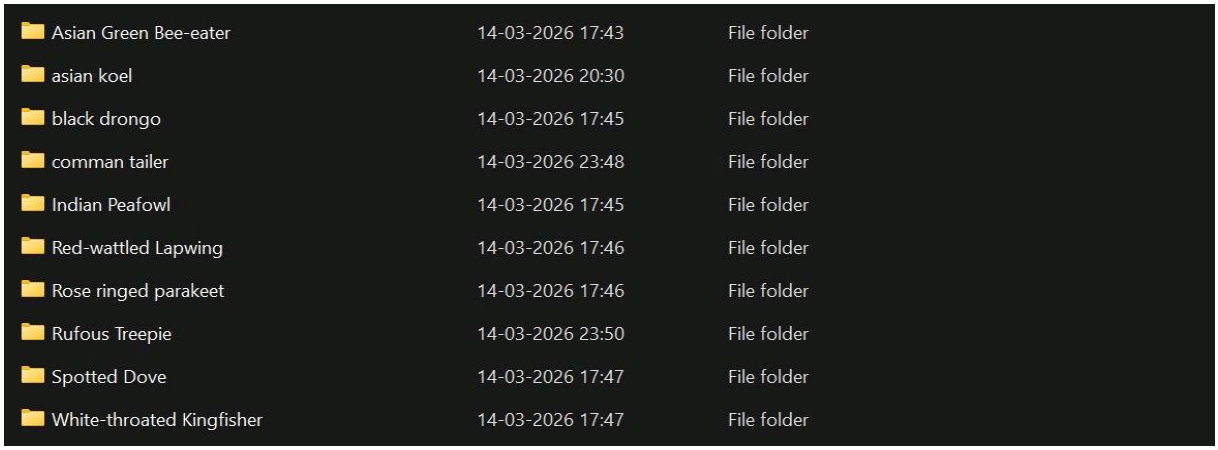


Figure 3.1 Merlin Bird App Data Set

As shown in Figure 3.1 Extracting isolated audio from the Merlin Bird App required a multi-step conversion protocol. The application interface was screen-recorded, and the resulting media was processed through conversion software to isolate the MP3 audio tracks. And delete the video. We sorted all those fresh MP3s into folders and then split the pile up. Most of the clips went directly into training the deep learning model, but we deliberately held a small chunk back so we could test how accurate the model was later on.

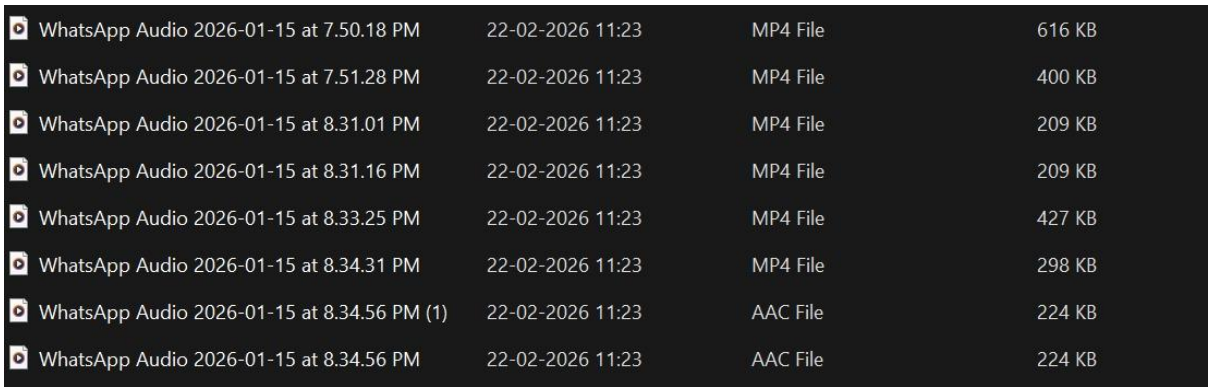


Figure 3.2 Screen-Recorded to MP3 Audio

As shown in figure 3.2, the sounds from the Merlin Bird App, we simply screen-recorded the calls on a smartphone. After moving them to a laptop, we converted those MP4 video files into clean audio tracks to train our model.

3.5.1.2 Xeno-Canto database

As shown in Figure 3.3, Xeno-Canto is a massive, open-access online database where people from all over the world upload and share wildlife recordings. For our project, it was incredibly useful because it provided hundreds of different audio samples for specific bird calls, giving our deep learning model plenty of varied data to learn from.



Name	#	Title	Contributing artists	Album
XC29590 - Asian Koel - ...		Asian Koel (Eudynamys scolo...		xeno-canto
XC29590 - Asian Koel - ...				
XC29594 - Asian Koel - ...		Asian Koel (Eudynamys scolo...		xeno-canto
XC31590 - Asian Koel - ...		Asian Koel (Eudynamys scolo...	Stuart Fisher	xeno-canto
XC31590 - Asian Koel - ...		Asian Koel (Eudynamys scolo...	Stuart Fisher	xeno-canto
XC35804 - Asian Koel - ...		Asian Koel (Eudynamys scolo...	Arnold Meijer	xeno-canto
XC36966 - Asian Koel - ...		Asian Koel (Eudynamys scolo...	Sander Pieterse	xeno-canto
XC37625 - Asian Koel - ...		Asian Koel (Eudynamys scolo...	John van der Woude	xeno-canto
XC37974 - Asian Koel - ...		Asian Koel (Eudynamys scolo...	Liao Xiao-dong	xeno-canto
XC54452 - Asian Koel - ...		Asian Koel (Eudynamys scolo...	Natalie Raeber	xeno-canto
XC76182 - Asian Koel - ...		Asian Koel (Eudynamys scolo...	vir joshi	xeno-canto
XC78432 - Asian Koel - ...		Asian Koel (Eudynamys scolo...	Frank Lambert	xeno-canto
XC78434 - Asian Koel - ...		Asian Koel (Eudynamys scolo...	Frank Lambert	xeno-canto
XC78435 - Asian Koel - ...		Asian Koel (Eudynamys scolo...	Frank Lambert	xeno-canto
XC90127 - Asian Koel - ...		Asian Koel (Eudynamys scolo...	Chie-Jen (Jerome) Ko	xeno-canto
XC93933 - Asian Koel - ...		Asian Koel (Eudynamys scolo...	Mike Nelson	xeno-canto

Figure 3.3 Xeno-Canto database Dataset

3.5.1.3 Manually recorded

As shown in Figure 3.4, to get real-world audio, Field recordings were collected across the campus and manually recorded the local birds. This gave us authentic training data that included all the natural background noises of the environment.

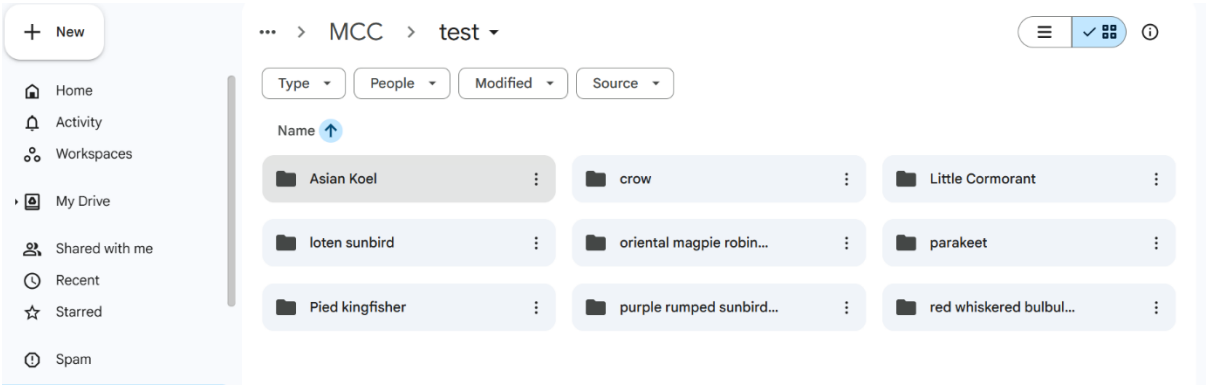


Figure 3.4 Manually recorded Dataset

3.5.2 Audio Standardization

As shown in Figure 3.5, the audio files had conflicting parameters. Files arrived in MP3 and WAV formats with arbitrary sample rates ranging from 44.1 kHz to 48 kHz. Neural networks require fixed-size multidimensional arrays, making this raw data unusable.

To fix this, we used a specific conversion tool and a custom Python script to handle the bulk processing of the files. We first utilized NoteBurner Music One to help manage and prepare the audio tracks from different formats. After that, our Python script performed several steps to standardize the data, including forcing every file into a single-channel mono WAV format. Stereo data was removed because it takes up twice the memory space without providing extra classification value. The sample rate was locked at exactly 16 kHz across the entire directory. Finally, the script chopped the long, continuous tracks into equal three-second segments and used volume limits to automatically delete any blocks that were just dead silence.

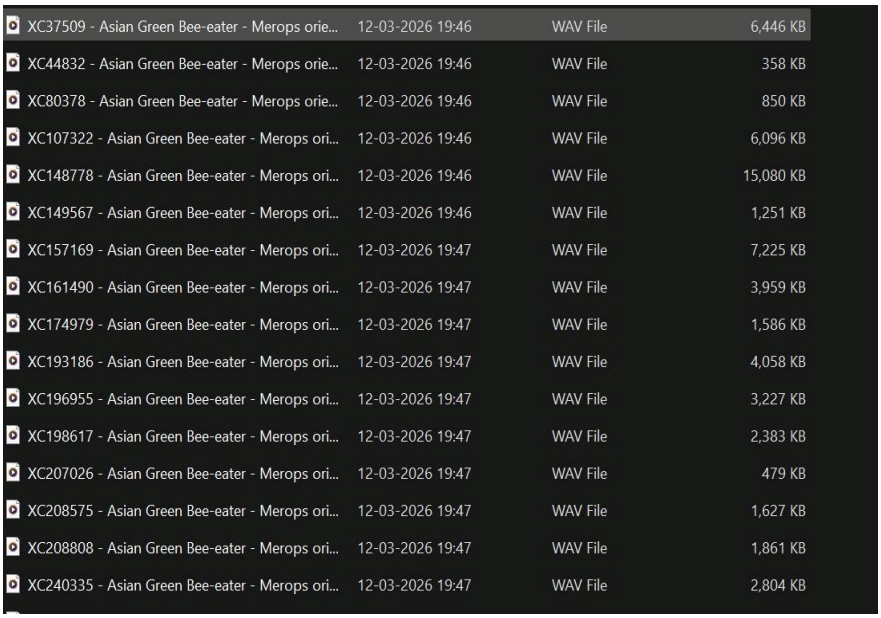


Figure 3.5 MP3 and WAV File Samples

3.5.3 Pre-processing using the MFC Model

Raw audio arrays are computationally expensive and difficult for a neural network to parse accurately. We applied the Mel-Frequency Cepstrum (MFC) technique to transform the one-dimensional sound waves into two-dimensional visual grids.

The Mel scale warps standard frequencies to map directly to human auditory perception, applying greater resolution to lower frequencies. The project specifically utilized the full MFC matrix rather than extracting isolated coefficients. Running the audio segments through the MFC algorithm produced a complete 2D grid representing frequency shifts over time. Generating this dense textural image allowed the data to be formatted perfectly for a spatial pattern recognition network.

3.6 Model Selection and Training Phase

Model compilation, training, and validation were executed inside Google Colab and Edge Impulse environments to access remote GPU clusters, bypassing the hardware limitations of a standard development machine.

3.6.1 Exploring Deep Learning Architectures

Three distinct neural network architectures were evaluated to determine the most effective approach for this project. Long Short-Term Memory (LSTM) networks were tested first due to their standard application in time-series data. Transformer models were also built to test their attention mechanisms against ambient background noise. Finally, a Convolutional Neural Network (CNN)was compiled to process the acoustic features of the bird calls directly.

3.6.2 The Rationale for Selecting CNN

During software testing, the LSTM and Transformer networks produced acceptable results but did not reach the high performance levels required for reliable field use. While these models showed potential, they ultimately struggled with the specific complexities of bird vocalizations in a live environment. Furthermore, the parameter counts and RAM requirements for both architectures exceeded the physical memory limits of the Arduino hardware, making them impractical for edge deployment.

The CNN was ultimately selected as the final architecture because it proved to be the best model for real-time audio prediction on a robotic IoT device. By passing convolutional filters over the acoustic matrices, the CNN demonstrated a superior ability to isolate the specific patterns of bird calls compared to alternative architectures. The mathematical efficiency of the pooling layers maintained the final parameter count small enough to flash onto the microcontroller without sacrificing any analytical power. This balance of superior precision

and compact size made the CNN the most robust and effective choice for automated monitoring.

3.6.3 Training and Conversion for Edge

The CNN architecture was designed as a sequential model to process acoustic data effectively. The initial stage involves a Reshape layer that organizes the input length into segments of 40, allowing the network to analyze the audio as structured frames. The core processing is handled by two Conv1D layers. The first layer utilizes 8 filters with a kernel size of 3 to capture fundamental frequency patterns, while the second layer increases to 16 filters to extract more complex bird call characteristics.

To maintain efficiency and prevent the model from overfitting, each convolutional block is followed by a MaxPooling1D layer with a pool size and stride of 2. These layers successfully reduced the data dimensionality, which maintained a low parameter count for the device. Additionally, Dropout layers set at 0.25 were integrated to ensure the model generalizes well to new, real-world sounds from the campus.

Once the accuracy goals were achieved through training with the Adam optimizer over the specified epochs, the model was prepared for the edge. Using the TensorFlow Lite for Microcontrollers framework, the model underwent post-training quantization. By converting 32-bit floating-point weights into 8-bit integers, the system drastically lowered the memory required for storage. This process resulted in a compact C++ header array that provides high-speed, reliable predictions directly on the Arduino hardware.

3.7 Hardware Setup and Edge Deployment

The physical build of this monitoring system was designed around a processing unit capable of executing embedded machine learning algorithms without drawing excessive current. To achieve this, the workload was split between two specific microcontrollers: the Arduino Nano 33 BLE and the ESP8266.

3.7.1 The Microcontroller: Arduino Nano 33 BLE

As shown in Figure 3.6, the Arduino Nano 33 BLE was designated as the core compute node. Standard 8-bit microcontrollers lack the processing speed required for live audio

classification. The Nano 33 BLE operates on a 32-bit Arm Cortex-M4F processor. The inclusion of a hardware floating-point unit (FPU) allows the board to execute the TinyML matrix multiplications natively without critical processing delays.

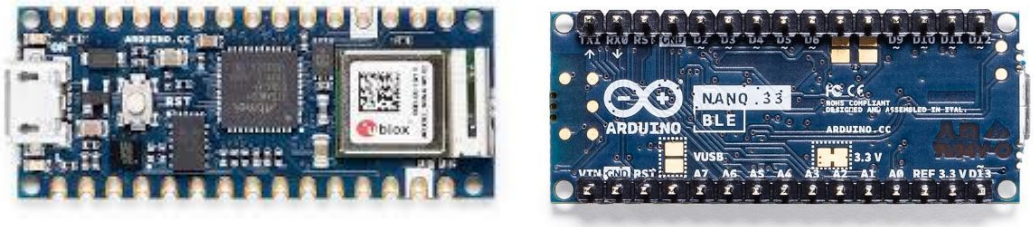


Figure 3.6 Arduino Nano 33 BLE

3.7.2 Audio Interfacing and Real-Time Inference

An analog microphone was wired into the ADC pins of the Arduino. The listening software relies on a continuous ring buffer. The microphone constantly overwrites the oldest memory addresses with new acoustic inputs.

The main loop extracts a window from this buffer multiple times per second. It instantly calculates the MFC grid and passes the array through the (TinyML) header file. The neural network outputs an array of confidence values. When the probability metric for a target bird species crosses the programmed threshold, the system triggers the serial transmission protocol.

3.7.3 IoT Integration and Cloud Storage

The final engineering step required pushing the local edge data to a centralized server. The Arduino Nano 33 BLE lacks native Wi-Fi hardware, requiring a secondary module to handle the network stack.

3.7.4 The Communication Bridge: ESP8266

An ESP8266 microcontroller was integrated into the circuit alongside the Arduino. As shown in Figure 3.7, this module handles the network connection. Data passes between the two

boards via UART hardware serial communication. The TX pin of the Arduino connects to the RX pin of the ESP8266, and the RX connects to the TX. As shown in Figure 3.8, jumper wires are used to establish these physical links.

A major design advantage of this specific hardware pairing is the voltage logic. Both the Arduino Nano 33 BLE and the ESP8266 operate on 3.3V logic levels. This permitted direct jumper wiring between the serial pins without requiring dedicated logic level shifting circuits, simplifying the physical footprint and reducing the risk of component failure. The baud rate was synchronized between both boards to ensure stable text transmission.



Figure 3.7 ESP8266



Figure 3.8 Jumper Wires

3.7.5 Cloud Database: Supabase

Upon a positive classification, the Arduino executes a standard serial write command. It pushes a plain text payload containing the bird species name across the UART lines.

The ESP8266 maintains a persistent background connection to the campus Wi-Fi. When the serial buffer receives the text from the Arduino, the ESP8266 packages the string into a JSON format. It triggers an HTTP POST request and transmits the JSON payload directly to a Supabase database. Supabase was chosen for the backend due to its low-latency PostgreSQL infrastructure.

The database schema captures the incoming species string and automatically appends the exact server timestamp and the sensor node's location ID. This architecture strips out all audio transmission, ensuring the logging system remains stable even in campus zones with poor wireless signal strength.

3.8 Overview Project

As illustrated in Figure 3.9, the system architecture establishes a seamless, automated pipeline that bridges the gap between field acoustics and remote data management. The process begins at the edge, where a specialized sensor node captures live bird calls through a high-sensitivity microphone. Instead of streaming heavy audio files which would exhaust battery and bandwidth the Arduino Nano 33 BLE processes these signals locally. It utilizes an embedded Convolutional Neural Network (CNN) to identify species patterns without requiring a constant cloud connection. Once a bird is recognized, the hardware transmits a lightweight text string to an ESP8266 Wi-Fi module via a serial link. This module acts as a network bridge, pushing the identification to a Supabase cloud platform where the detection is permanently archived. The final result is a real-time monitoring dashboard accessible via smartphones or laptops, providing researchers with an immediate, data-driven view of avian activity within the Madras Christian College ecosystem.

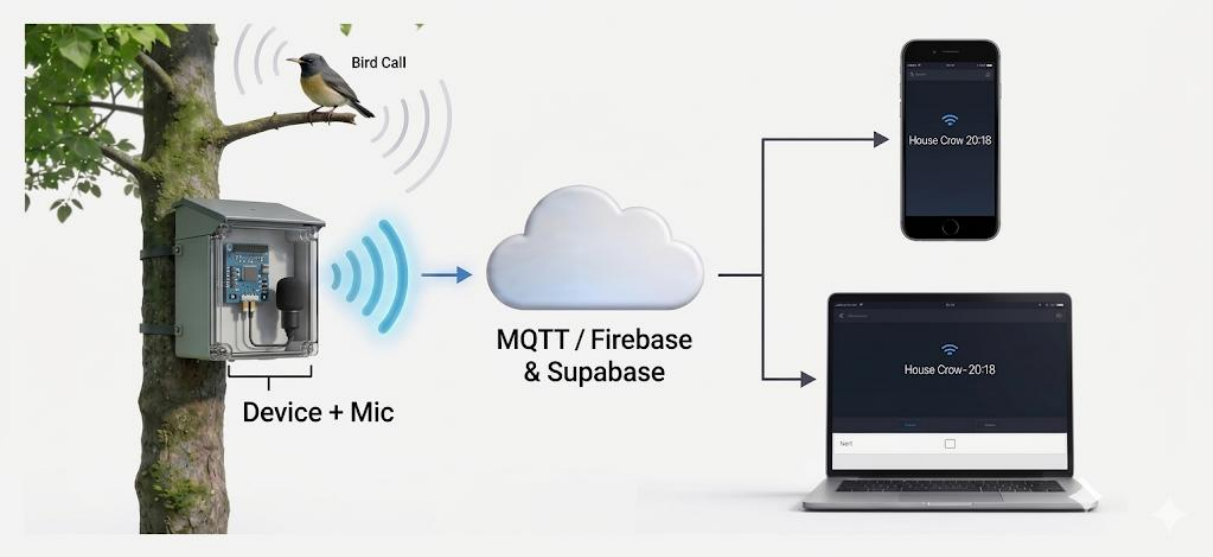


Figure 3.9 Overview Project

4.HARDWARE AND SOFTWARE IMPLEMENTATION

4.1 Introduction

Once the deep learning models were trained on our computers, we had to figure out how to make them work in the real world. This chapter breaks down the actual physical build of our bird monitor. This chapter details the specific microcontrollers selected and outlines the breadboard wiring schematic. on the breadboard. On top of the hardware, I'll also walk through the coding process. That includes getting the AI model small enough to fit on an Arduino and making sure our Supabase database actually caught the data over Wi-Fi.

4.2 Hardware Components Used

Building the physical prototype was an exercise in strict budget and power management. Buying expensive, pre-made acoustic data loggers wasn't an option for this project. We needed to keep the build costs completely minimized while focusing heavily on saving battery life for outdoor deployment. Instead of trying to force a single piece of hardware to do everything, we ended up splitting the workload across three specific, highly affordable components to make the system as efficient as possible.

4.2.1 Arduino Nano 33 BLE (The Processing Core)

The core processing unit for the entire system is the Arduino Nano 33 BLE. We specifically needed a microcontroller that was physically tiny but packed enough raw computing power to run embedded machine learning models without draining a portable battery in a few hours. This specific board runs on a 32-bit ARM Cortex-M4 processor, which makes it uniquely capable of handling the heavy matrix math required by our Convolutional Neural Network. This microcontroller functions as the central processing unit of the system. Its sole job is to sit in a continuous loop, sample the incoming audio, and run the MFC spectrogram conversions and CNN predictions completely offline. Because the board only has 1MB of flash storage and 256KB of RAM, we had to aggressively compress our AI model to make it fit, but it perfectly handles the localized edge computing we required.

4.2.2 ESP8266 Wi-Fi Module (The Network Messenger)

The ESP8266 Wi-Fi module. During early testing, we quickly realized that forcing a single microcontroller to constantly listen to audio while simultaneously trying to maintain a stable Wi-Fi connection caused massive system crashes and dropped audio frames. The Arduino was already working at its absolute maximum capacity just doing the AI prediction math. To solve this bottleneck, we brought in the ESP8266 to act strictly as a dedicated, cheap internet messenger. We wired the two boards together using a basic UART serial connection. Now, the workload is split perfectly: the Arduino does all the heavy thinking, and the moment it successfully identifies a bird, it just passes a simple text string over to the ESP8266. The Wi-Fi module grabs that text, wakes up, and handles all the complex HTTP requests to push the data securely to our remote Supabase server.

4.2.3 Analog Microphone Module (The Acoustic Sensor)

Finally, to actually capture the environmental sound, we wired up a standard external analog microphone module to act as the device's ears. While some modern microcontrollers feature tiny built-in microphones, those are usually designed for close-range voice commands like a smartwatch, not for picking up faint bird calls high up in the campus trees. By using a dedicated external analog module, we gained the ability to manually adjust the hardware gain using the onboard potentiometer. This allowed us to physically boost the sensitivity of the microphone in the field, ensuring the audio waves continuously feeding into the Arduino's analog-to-digital converter were loud and clear enough for the neural network to actually process and recognize.

4.3 Circuit Design and Wiring

The wiring configuration required precision. Integrating two distinct microcontrollers necessitates careful routing of communication lines. About how they talk to each other so the data doesn't get messed up. As shown in Figure 4.1, the physical connections were made to ensure stability between all components.

We started with the microphone. We took the power and ground pins from the mic and wired them directly to the 3.3V and GND pins on the Arduino Nano. The actual sound output

wire from the microphone plugged straight into an analog input pin. This lets the Arduino continuously read the physical sound waves.

Connecting the Arduino to the ESP8266 was the next step, and we used basic serial communication for this part. The trick here is that the transmit wire (TX) from one board always has to cross over to the receive wire (RX) of the other. So, the Arduino's TX went to the ESP8266's RX, and in reverse order. The most important part was plugging their ground pins together. If the boards don't share a common ground wire, the electrical signals turn into gibberish, so we made sure that was completely secure.

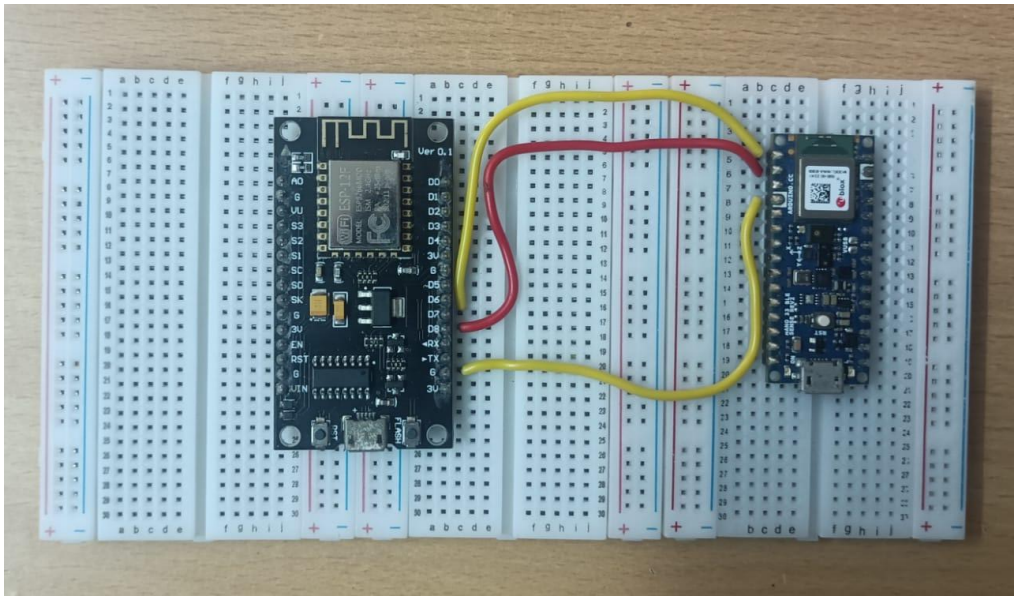


Figure 4.1 Circuit Design and Wiring

Wiring the physical prototype required a highly specific breadboard layout. If the power and data lines are not routed perfectly, the two microcontrollers will fail to communicate or potentially short out. The electrical connections were organized through the following steps:

- **Main Power Delivery:** The entire system gets its electricity through a basic USB connection. A cable running from a laptop or portable battery plugs directly into the primary Arduino, pushing the required operating voltage into the main board to keep the sensors active.
- **Shared Ground Connection:** Electronic logic signals get completely corrupted if different boards do not share the exact same voltage floor. Tying the system together

requires a jumper wire running straight from the ground (GND) pin of the Arduino over to the ground (GND) pin on the ESP8266.

- **Serial Transmit Path:** Sending data out requires crossing the hardware communication wires. The text payload leaves the Arduino through its transmit (TX) terminal, so a cable routes that electrical signal directly into the receiving (RX) terminal on the Wi-Fi module.
- **Serial Receive Path:** The circuit loop closes by wiring the opposite pins together. The receive (RX) pin on the Arduino is hooked up directly to the transmit (TX) pin on the ESP8266. Flipping these communication lines is an absolute requirement; otherwise, the text packets cannot travel back and forth without turning into unreadable errors.

4.4 Software Environment and Tools

To actually write the scripts and flash our code onto the hardware, we stuck to a few specific tools. Everything was programmed using the standard Arduino IDE. It's free and works perfectly for writing scripts for both the Nano 33 BLE and the ESP8266.

The biggest software challenge was dealing with the CNN model. Models trained in desktop environments generate file sizes that exceed standard microcontroller storage limits. To solve this, we ran the file through TensorFlow Lite for Microcontrollers. This tool basically compresses the massive AI model into a long C++ byte array. After compressing it, we integrated the array into that massive block of numbers into our Arduino code, which allowed the board to do offline predictions.

4.5 Implementation of the Workflow

Because we used two boards, we had to write two completely separate scripts that run at the same time.

As shown in Figure 4.2, the Arduino side, the code runs in an endless loop. Every few seconds, it samples a chunk of audio from the mic. It extracts the features we need and runs the math through that compressed CNN model. If it detects a matching acoustic signature say, a Asian koel it simply takes that bird's name and prints it out as plain text through its TX wire.

The ESP8266 code handles the networking side. When you turn it on, it immediately logs into the campus Wi-Fi using the credentials we coded into it. After connecting, it just waits and watches its RX pin. The exact moment it sees a bird name come down the wire from the Arduino, it grabs that text, builds an HTTP POST request, and shoots it off to the internet.

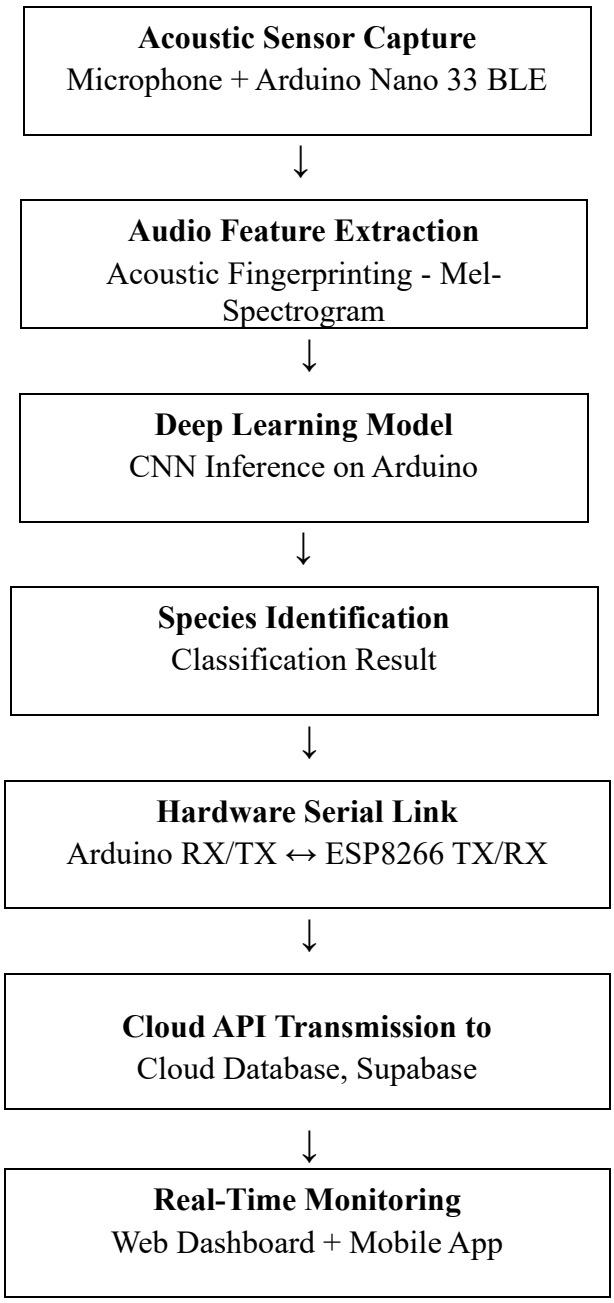


Figure 4.2 Implementation of the Workflow

4.6 Detailed Code Logic

The technical success of this monitoring system relies on a seamless transition from raw acoustic capture to cloud-based logging. By offloading specific tasks to dedicated microcontrollers, the code ensures that high-speed neural network predictions and stable Wi-Fi transmissions occur simultaneously without system latency. This chapter examines the specific scripts that govern the internal "intelligence" of the Arduino, the communication protocols between boards, and the final secure handshake with the Supabase database.

4.6.1 Deep Learning Model & Training (TensorFlow)

The initial phase of the software development involves defining the core intelligence of the system using a Convolutional Neural Network (CNN). Built using the TensorFlow and Keras frameworks, the architecture is specifically structured to handle 1D audio signals by first applying a Reshape layer that organizes the input length into fixed frames of 40. The model then utilizes two sequential Conv1D layers, starting with 8 filters and increasing to 16, to extract the specific frequency patterns and acoustic traits unique to each bird species. To maintain efficiency and ensure the model generalizes well to real-world sounds, MaxPooling1D layers reduce the data dimensionality, while Dropout layers (0.25) prevent the network from overfitting. The entire network is compiled with the Adam optimizer and trained to minimize categorical cross-entropy, effectively turning raw sound segments into a highly accurate probability distribution for live classification.

4.6.2 Real-Time Edge Inference (Arduino Nano 33 BLE)

Once the model is trained and deployed via Edge Impulse, the Arduino Nano 33 BLE Sense acts as the sensor node. This code manages the PDM (Pulse Density Modulation) microphone to capture live audio. It uses a double-buffering technique (via inference_t) to ensure that while the processor is analyzing one segment of audio, the microphone can continue recording the next without any gaps. When the run_classifier_continuous function identifies a bird sound with a confidence score above 0.70 (70%), it selects the highest-probability label. This result is then sent out physically through the Serial1 (UART) pins to the communication module.

4.6.3 Data Transmission & Cloud Integration (ESP8266 & Supabase)

The final stage involves the ESP8266 (NodeMCU), which serves as the bridge between the hardware and the internet. It receives the bird species name from the Arduino via SoftwareSerial. Once a name is received, the ESP8266 synchronizes with an NTP server to get the current India Standard Time (IST) and date. It then packages this information the bird name, the timestamp, and the specific MCC campus location into a JSON payload. This data is sent via a Secure HTTPS POST request to your Supabase REST API. By providing the API key and Bearer token in the header, the system securely logs every detection into your database in real-time, making the bioacoustic data accessible from anywhere in the world.

4.7 Supabase Database Setup

For the backend, we went with Supabase. It is an open-source platform based on PostgreSQL, and it gave us instant API access without having to build a custom server from scratch.We created a table called bird_detections with exactly four columns, which can be seen in the database layout in Figure 4.3:

1. **id:** An auto-incrementing number for each record.
2. **bird name:** To store the text of the species detected.
3. **location:** To store the geographic tag sent by the ESP8266.
4. **detected time:** A timestamptz column set to default to now ().

Because of that last column, the Supabase server automatically stamps the exact time the data arrives. We didn't have to program the hardware to track the time. Whenever a bird is detected, the ESP8266 hits the Supabase API with our project key, and the data is logged instantly. As shown in Figure 4.3, this gives us a live dashboard where we can see exactly what birds are present, where they are, and at what time.

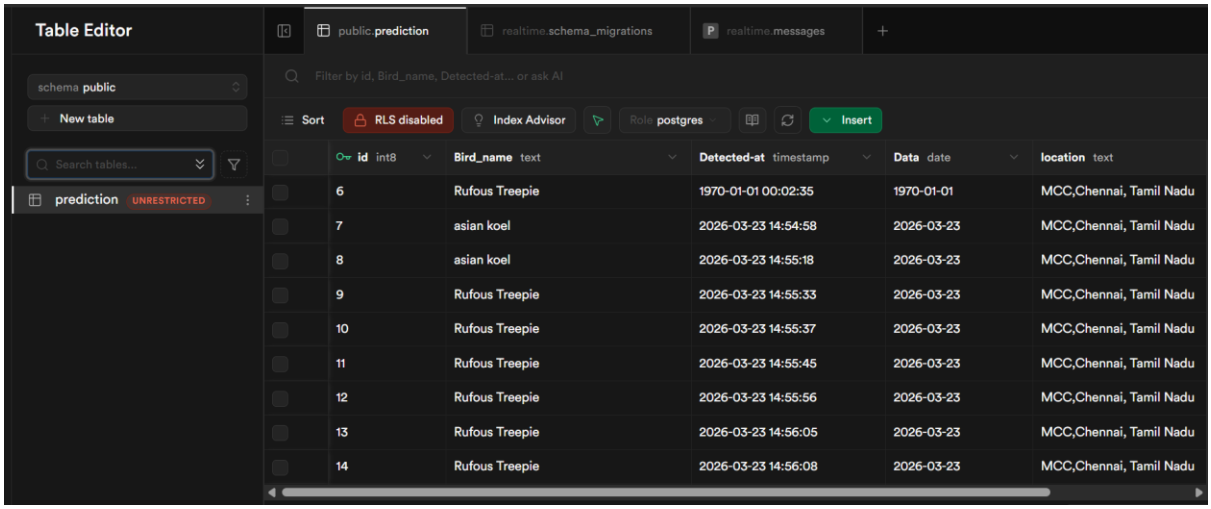


Figure 4.3 Supabase Database Setup

5. RESULTS AND DISCUSSION

5.1 Introduction

Following hardware assembly and software integration, the final phase involved field testing the prototype in a live outdoor environment. And into the real world. I will cover how the CNN model performed during its training phase, how the physical Arduino handled the outdoor audio processing, and whether or not the ESP8266 successfully pushed our data to the Supabase cloud. Finally, I will discuss what went right, what went wrong, and the overall limits of our build.

5.2 Model Training and Software Results

Before moving our project onto the hardware, we had to run extensive tests on a computer to see which deep learning model could actually tell different bird species apart in a real-world setting. Selecting the right software architecture was a difficult balancing act because we needed high accuracy, but we were limited by the very small memory space on the Arduino Nano 33 BLE. We tested four main models: a standard CNN, a standalone LSTM, a Transformer, and a hybrid CNN+LSTM. Our main goal wasn't just to get a high score on a desktop, but to find a model that worked fast enough to identify birds in the middle of noisy campus environments.

Since audio changes over time, we first looked at sequential models like LSTMs and Transformers, but they were not practical for this specific IoT device. The LSTM struggled to decode the complex patterns in our audio data, leading to unstable results. While the Transformer is popular for audio, it required far too much computing power. When we tried to shrink these models down to fit the Arduino, the accuracy dropped significantly, and they were too slow for live listening. These architectures took up too much storage space and the prediction time was way too high for a real-time system.

In the end, the standalone CNN was clearly the best choice for our project. It achieved the highest predictive accuracy at 93% and performed much faster than the other architectures. Unlike the LSTMs that try to follow a timeline, the CNN treated the audio features as patterns, which allowed it to be incredibly lightweight and efficient. Because it takes up very little space and processes data almost instantly, it provides the lightning-fast, reliable predictions we need

for a robotic monitoring device. This efficiency is why we chose the CNN as our permanent model for the final hardware deployment.

As shown in Figure 5.1. The data in the performance table clearly shows why the CNN was the final choice for this project. While hybrid models like CNN+Transformer reached 83.72% accuracy, they were simply too heavy for the Arduino's limited memory. The standalone CNN hit a peak accuracy of 93.00% while staying lightweight enough for real-time use. Other models, like the LSTM at 74.00% and the Transformer at 70.00%, couldn't match this precision or the speed needed for live deployment. This high accuracy and small size made the CNN the only practical option.

Model Architectures	Accuracy (%)
CNN	93.00%
LSTM	74.00%
Transformer	70.00%
CNN + LSTM	81.75%
CNN + Transformer	83.72%

Figure 5.1 Model Accuracy Comparison Table

As shown in Figure 5.2, the performance chart, the standalone CNN clearly outperformed the other models with a peak accuracy of 93%. While hybrid architectures like CNN+Transformer reached 83.72%, they proved too computationally heavy for the Arduino's limited memory. Sequential models like the LSTM and Transformer fell behind, achieving only 74% and 70% respectively. The CNN’s high precision and lightweight design make it the most effective choice for real-time monitoring.

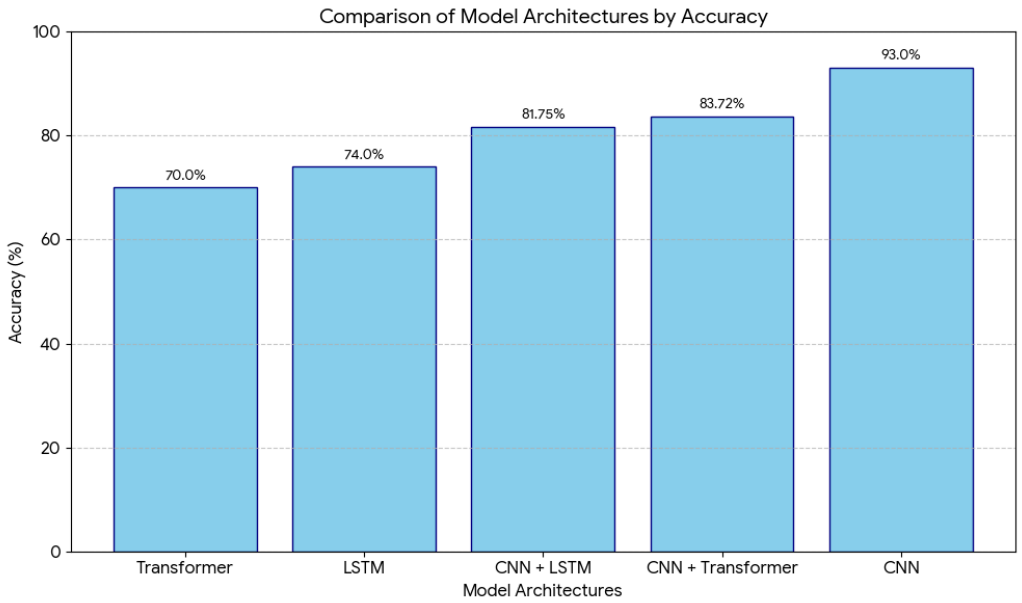


Figure 5.2 Accuracy Chart

As shown in Figure 5.3 The Confusion Matrix confirms that our CNN model achieved excellent precision, with most bird species scoring over 90% accuracy. The Red-wattled Lapwing reached the highest individual success rate at 98.5%. While the Asian Koel had slightly more difficulty at 88.5%, the overall F1 scores remain strong across all categories. This high-level performance proves the CNN is perfectly capable of identifying specific bird calls with minimal errors, making it a reliable tool for our automated monitoring system.

Confusion matrix (validation set)

	ASIAN GR	RED-WATT	RUFOUS T	ASIAN KO	COMMAN
ASIAN GREE	**97.0%**	1.7%	0.9%	0%	0.4%
RED-WATTLE	0%	**98.5%**	1.5%	0%	0%
RUFOUS TRE	2.0%	1.2%	**94.9%**	2.0%	0%
ASIAN KOEL	0.9%	5.1%	5.1%	**88.5%**	0.4%
COMMAN TA	1.8%	7.9%	0.4%	0.9%	**89.0%**
F1 SCORE	0.96	0.91	0.94	0.92	0.94

Figure 5.3 Confusion Matrix

As shown in Figure 5.4, the training curve illustrates a steady increase in model accuracy as the training progressed through the 50 epochs. Starting from a lower baseline, both the training and validation accuracy climbed rapidly before stabilizing above the 90% mark. Although there is a visible dip around epoch 33, the model quickly recovered and maintained high performance. This consistent growth proves that the CNN was effectively learning the patterns in the bird sound data without suffering from significant overfitting. Final results show the model reached a peak accuracy of 93%, confirming its reliability for real-time monitoring.

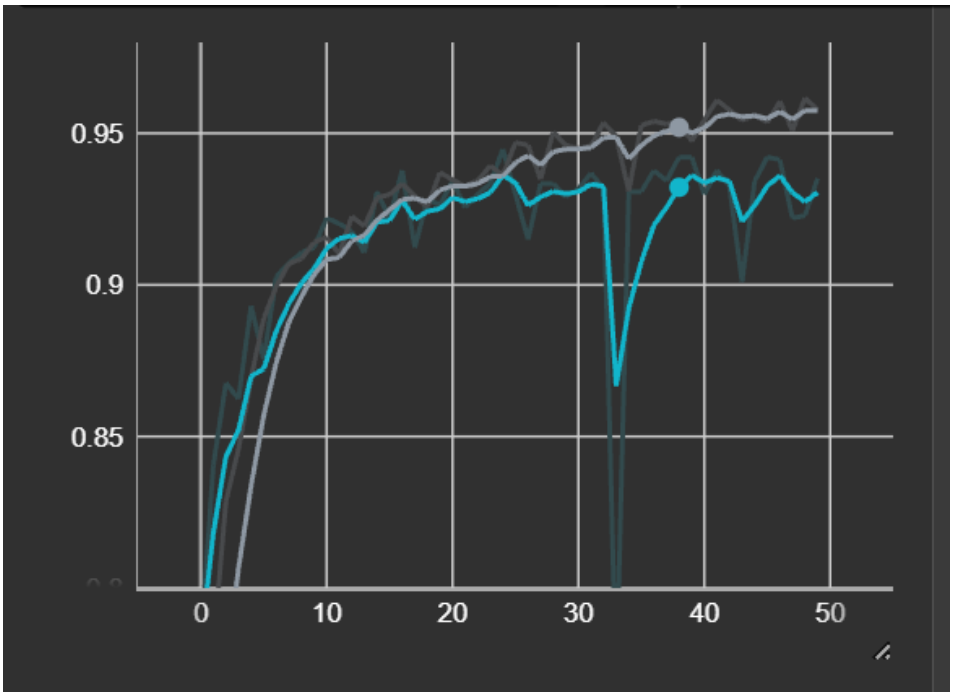


Figure 5.4 Training Accuracy Curves

As shown in Figure 5.5, the loss curve clearly demonstrates how the training error decreased as the model progressed through 50 epochs. In the early stages, the loss values dropped sharply as the CNN learned to identify the unique frequency patterns of the bird calls, eventually leveling off as the network reached a stable state. Although there was a temporary spike around epoch 33 which corresponds exactly with the dip in the accuracy graph the system quickly corrected itself and continued to improve. This consistent downward trend confirms that the Adam optimizer effectively minimized categorical cross-entropy throughout the process. By the end of the training phase, the loss reached a minimal level, proving that the model is highly reliable for identifying target bird species in the field.

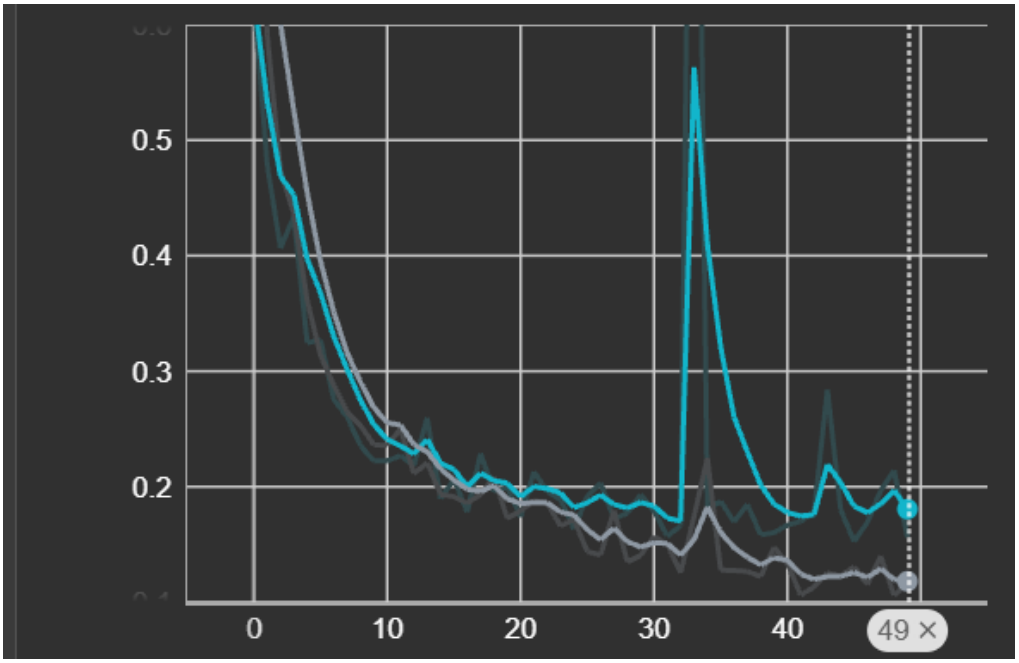


Figure 5.5 Training Loss and Accuracy Curves

5.3 Hardware and Real-Time Testing.

The true test of the project was seeing if the hardware could handle live audio outdoors. The physical hardware prototype was deployed on the campus grounds to evaluate live acoustic capture.

The Arduino Nano 33 BLE handled the live audio processing surprisingly well. The microphone constantly fed sound waves into the board, and the Arduino chopped the audio into short segments. We connected a laptop to the board just to watch the serial monitor and see internal processing logic in real time.

When a bird made a noise, we watched the Arduino process the MFC features and run the CNN math. Within about one or two seconds, the board would generate an inference. For example, during one of our tests, a Koel started cawing nearby. A second later, the serial monitor printed "Asian Koel" with a high confidence score. Seeing the board successfully grab a live sound, filter it, and accurately identify the species entirely offline was the biggest success of the project.

5.4 Cloud Integration and Network Results

Having the Arduino accurately classify the species is only half the project; the other half is getting that information to the internet. We left the prototype outside and went inside to monitor the Supabase database on a laptop.

This is where the ESP8266 came in. The Wi-Fi module managed to connect to the local network without any issues. Upon successful species classification by the edge device, the ESP8266 transmission indicator illuminated, meaning it was sending the HTTP POST request. Almost instantly, our Supabase dashboard would update.

We watched the bird_detections table fill up with live data. A new row would pop up showing the ID number, the text "Asian Koel" and the exact timestamp, like "20:18." This proved that our dual-microcontroller setup worked perfectly. By separating the heavy AI math from the Wi-Fi tasks, neither board crashed, and the data flow was completely stable. It also proved how data-efficient the system is. Instead of uploading heavy WAV files that take minutes to send, we were only sending a few bytes of text, meaning the live feed updated almost instantly.

5.5 Real-Time System Operation and Live Results

The final testing phase evaluated the complete hardware-to-cloud pipeline during live acoustic events. The system workflow operates in three distinct physical stages to ensure stability. First, the Arduino Nano 33 BLE acts as the localized prediction engine, running the deep learning model offline to identify the avian species. Second, upon a positive identification, the resulting text string is pushed across the RX and TX hardware serial wires to the ESP8266 module. Finally, the ESP8266 acts as a network bridge, utilizing the local Wi-Fi to transmit that text payload directly to the Supabase database for permanent storage.

5.5.1 Live Edge Prediction (Arduino Nano 33 BLE)

During field deployment, the primary microcontroller successfully handled the continuous audio buffering and model inference without system failures. When a target bird vocalized near the sensor, the embedded Convolutional Neural Network processed the acoustic features and identified the specific pattern. As shown in Figure 5.6, the Arduino immediatel

output the correct species name to the local serial port. This successful edge classification verified that the compressed model functions accurately in a real-world environment, isolating the target audio from background wind and campus noise.

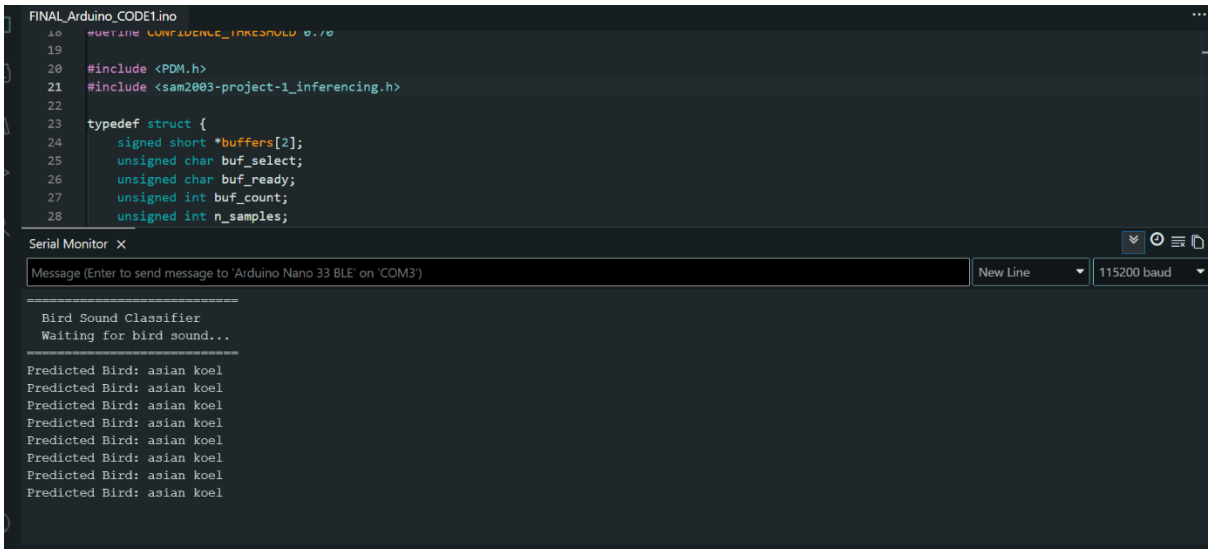


Figure 5.6 Live Edge Prediction (Arduino Nano 33 BLE)

5.5.2 Hardware Serial Routing (ESP8266 Module)

The intermediate communication bridge performed exactly as designed. The ESP8266 maintained a persistent connection to the local internet router while continuously monitoring its hardware receive (RX) pin. As shown in Figure 5.7, the moment the plain text data arrived from the Arduino's transmit (TX) pin, the ESP8266 packaged the string into a structured JSON format. Because the physical workload was divided, the ESP8266 was able to focus entirely on formatting the HTTP POST request. This ensured the transmission occurred instantly without dropping data packets or crashing the local listening loop.

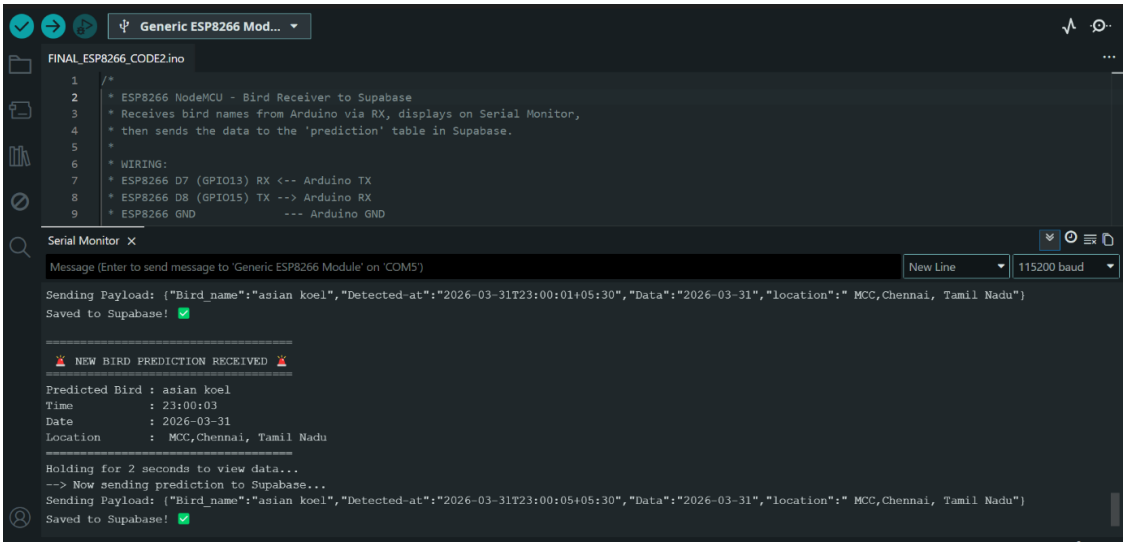


Figure 5.7 Hardware Serial Routing (ESP8266 Module)

5.5.3 Remote Database Logging (Supabase Platform)

The final validation of the system was confirming the cloud storage pipeline. By monitoring the Supabase backend during the live outdoor tests, the incoming data was verified in real-time. As shown in Figure 5.8, whenever the ESP8266 transmitted a payload over the Wi-Fi network, a new row automatically populated within the project database. The platform successfully captured the plain text bird name and immediately applied an accurate, automated server-side timestamp to the event. This resulted in a clean, chronological digital log of campus wildlife activity that required zero manual human intervention.

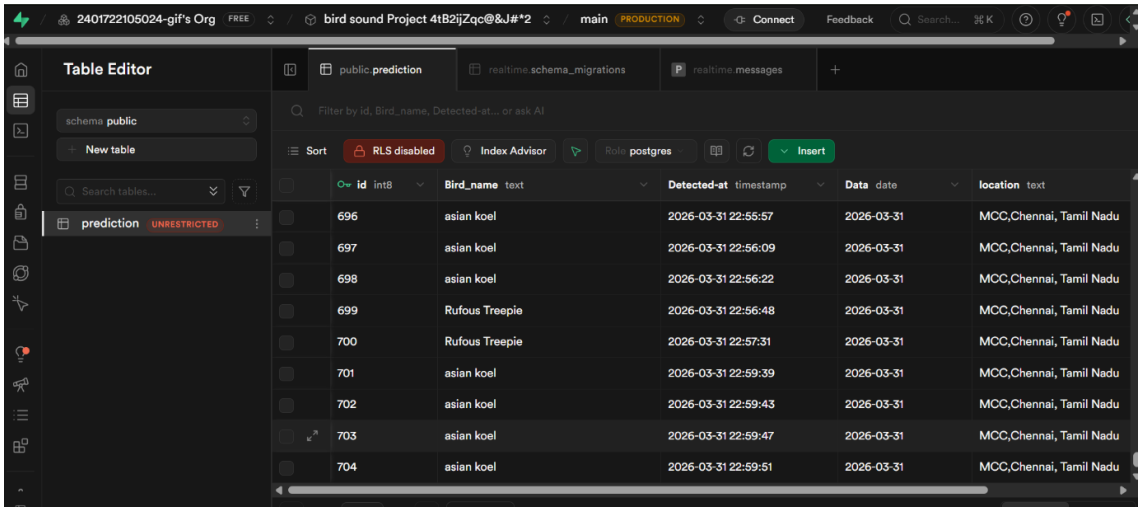


Figure 5.8 Remote Database Logging (Supabase Platform)

5.6 Discussion and System Limitations

Looking at the overall results, the project successfully solved the problem we started with. We built an automated system that can sit outside, identify birds by their sounds, and log them into a database so humans don't have to sit around watching them. However, taking the project into the real world definitely exposed a few limitations that we had to deal with.

The first major issue was environmental noise. Microphones pick up everything. If the wind was blowing hard, or if a group of students was talking loudly near the sensor, the audio got messy. Sometimes the CNN model would get confused by the wind hitting the microphone and fail to recognize a bird that was singing right next to it. Putting a foam cover over the microphone helped with the wind, but background noise is definitely the biggest challenge for acoustic monitoring.

The second limitation was the Wi-Fi range. The ESP8266 is a great little chip, but it doesn't have a massive antenna. If we placed the prototype too far into the trees, it would drop the connection to the campus Wi-Fi. The Arduino would still correctly identify the bird, but the ESP8266 couldn't send the text to Supabase until we moved it closer to a router.

Despite these small hardware limits, the results were highly positive. The prototype proved that edge computing running deep learning models directly on cheap microcontrollers is completely possible and highly effective for wildlife tracking. We successfully replaced a slow, manual observation method with a fast, digital, and automated IoT solution.

6. CONCLUSION

Deploying the automated acoustic monitor successfully addressed the core hypothesis of this research. can basic microcontrollers handle heavy machine learning? The successful field deployment proved that we do not need to rely on massive, expensive cloud computing servers to perform complex audio classification. By forcing the actual predictive math directly onto the edge hardware, the project validated a highly efficient approach to remote environmental monitoring.

Traditional audio tracking systems often fail because they attempt to stream live, continuous sound files to a remote server for analysis. That method constantly chokes network bandwidth, drains batteries rapidly, and introduces severe latency delays. Our architecture bypassed those bottlenecks entirely. Because all the audio processing happens locally on the device, the system never has to transmit a single heavy audio file over the internet.

The software backbone a standalone Convolutional Neural Network was the key to making this localized processing work. Running directly on the Arduino's logic board, the CNN analyzed the local Mel-Frequency Cepstrum (MFC) spectrograms the moment the microphone captured them. Despite the strict physical memory limits of the edge device, the model proved highly reliable. It successfully spotted our specific target bird calls in real-time by treating the incoming sound waves like static images, instantly finding the correct visual acoustic patterns hidden in the background noise.Once the Arduino made a successful identification, the ESP8266 Wi-Fi module took over to handle the network routing. This dual-board setup created a seamless, automated pipeline straight into the remote database. Instead of struggling to upload audio recordings, the ESP8266 only had to send a tiny, lightweight text string containing the identified bird's name and the exact timestamp. This kept the internet traffic minimal and ensured the text data was logged securely to the Supabase tables without dropping packets.

Ultimately, this physical prototype hit every single one of its initial engineering benchmarks. It successfully listened to a live environment, identified the correct acoustic patterns, and logged the formatted data to a remote server without ever needing a human sitting at a computer to supervise it. The system simply runs its loop autonomously. It stands as a highly functional, reliable proof-of-concept, clearly demonstrating that low-cost, low-power edge devices are entirely capable of handling serious, scalable wildlife tracking.

7.FUTURE ENHANCEMENT

Future Scope: Hardware Scaling and Broad Environmental Deployment

While the current breadboard setup proves that the core concept actually works, we are quickly hitting the absolute ceiling of what that tiny Arduino Nano microcontroller can process. The most glaring bottleneck in the current architecture is the severe lack of physical flash memory. Due to strict hardware memory limits, the trained CNN model required aggressive quantization and compression. just to flash it onto the board. This extreme compression means the system only has enough memory to recognize a handful of specific bird calls before it simply runs out of room.

The most critical upgrade moving forward is transitioning the primary processing hardware to an advanced model of the Raspberry Pi. A Raspberry Pi does not act like a basic microcontroller; it is a fully functional mini-computer. Upgrading to a Raspberry Pi give the system gigabytes of working memory rather than kilobytes. With that massive jump in computing power, the deep learning model no longer needs to be shrunk down. Advanced hardware would permit the deployment of a highly complex neural network trained on thousands of different bird calls, allowing the system to instantly identify almost any bird that flies past the microphone without lagging or crashing.

Furthermore, moving to a Raspberry Pi completely simplifies the physical electronics. Since a Pi has native Wi-Fi built directly into the board, we would no longer need the ESP8266 or the complicated serial wiring to bridge the internet connection. One single board would handle the audio capture, the AI prediction, and the cloud routing. It also features standard USB ports, which solves another major issue: the microphone. The onboard microphone on the Arduino is fine for close-range testing, but in a future build, we could plug a high-fidelity, weatherproof acoustic microphone directly into the Pi to pick up faint bird sounds from hundreds of feet away.

To push this system beyond the boundaries of the MCC campus, the physical housing and power delivery need a major redesign. Right now, the prototype relies on portable power banks, which eventually die and require manual swapping. A future deployment intended for heavy-duty, real-world environmental tracking needs to be entirely self-sufficient. By wiring a small solar charge controller and a lithium-ion battery pack into the circuit, and sealing the

entire rig inside a NEMA-rated waterproof enclosure, these autonomous listening nodes could be mounted to trees and left alone for months.

Deploying a grid of these solar-powered boxes deep into dense forests, wildlife reserves, or managed zoo environments opens up incredible possibilities for data collection. Researchers wouldn't have to sit in the woods for hours waiting to hear a specific bird. They could simply log into the web dashboard and look at the timestamps to track the morning dawn chorus, monitor seasonal migration shifts, or see exactly how local wildlife reacts to sudden climate changes.

Finally, the scope of this tracking system does not need to be restricted strictly to avian research. The underlying CNN algorithm doesn't actually know it is listening to a bird; it is simply looking for mathematical pattern matches inside an image-based spectrogram. Because of this, the hardware and software pipeline is completely universal. By simply swapping out the original training dataset, the model could easily be retrained to track the vocalizations of specific terrestrial animals, frogs, or endangered nocturnal mammals. We could even train it to listen for the sound of chainsaws or off-road vehicles to serve as an early warning system against illegal logging and poaching. Ultimately, upgrading the core hardware transforms this project from a simple, localized bird sound predictor into a highly adaptable acoustic tracking tool for global conservation efforts.

8.REFERENCES

Ashish Kumar Pandey, Sanyam Kudale, Rohit Jadhav, Swaraj Giri, S. K. Wagh (2025), Bird Species Prediction Based on Voice Using CRNN, International Journal for Research in Applied Science and Engineering Technology (IJRASET), vol.13, no.5, *https://www.ijraset.com/research-paper/bird-species-prediction-based-on-voice-using-crnn*

Deepak T. Mane, Pravin Kumar Bhoyar, Govinda B. Sambare, Gitanjali S. Mate, Gitanjali S. Mate, Archana Jadhav, Dipali Himmatrao Patil. (2026.) Deep learning-based bird sound classification using customized convolutional neural networks, Communications in Statistics: Case Studies, Data Analysis and Applications. (Published online 06 Jan 2026).

https://www.tandfonline.com/doi/full/10.1080/23737484.2025.2605342?scroll=top&needAccess=true

Indumathi C P, Diviya lakshmi K R and Mahalakshmi R (2024), Bird Sound Identification System using Deep Learning. Procedia Computer Science, vol.233, pp. 597–603.

http://www.sciencedirect.com/

Kartik Patil, Mallamma V Reddy (2024), Bird Sound Identification Using Machine learning, International Journal of Research Publication and Reviews, vol.5, no.10, pp.1608–1613. *https://ijrpr.com/uploads/V5ISSUE10/IJRPR33992.pdf*

Shubham Revadekar, Vidhish Panchal, Pratik Kanani, Kamal Shah, Anil Vasoya, Ravikumar Pandey (2023) Bird Sound Classification using Deep Neural Networks: A Comparative Analysis of State-of-the-Art Models and Custom Architectures, International Journal of Intelligent Systems and Applications in Engineering, vol. 11, no. 4, pp. 614–622, ISSN: 2147-6799.

.*https://ijisae.org/index.php/IJISAE/article/view/3596*

Sinduran Sivarajan (2016), Bird Sound Classification, B.E. dissertation, Dept. Comput., Imperial College London, London, U.K.

.*https://www.imperial.ac.uk/media/imperial-college/faculty-of-engineering/computing/public/SINDURANSIVARAJAN*.

Agus Kurniawan (2021), IoT Projects with Arduino Nano 33 BLE Sense: Step-By-Step Projects for Beginners, Apress, Berkeley, CA. *https://doi.org/10.1007/978-1-4842-6458-4*

Jeenali Shah, Rutuja Phalle, Vaishnavi Renke, and Dr. Pravin Gawande (2025), Edge driven Arduino Nano 33 BLE Sense in Modern Embedded Applications, Journal of Emerging Technologies and Innovative Research (JETIR), vol. 12, no. 11, pp. c589-c593.*www.jetir.org/view?paper=JETIR2511272*

Viswanatha V, Ramachandra A.C, Raghavendra Prasanna R, Prem Chowdary Kakarla, Viveka Simha PJ, and Nishant Mohan (2022), Implementation of Tiny Machine Learning Models on Arduino 33 BLE for Gesture and Speech Recognition, Journal of Xi'an University of Architecture & Technology, vol. XIV, issue 7, pp. 160-169. *https://arxiv.org/pdf/2207.12866*

Visan and E.M. Diaconu (2021), Home Automation System Using ESP8266 Microcontroller and Blynk Application, Valahia University of Targoviste, Faculty of Electrical Engineering, Electronics and Information Technology.

https://www.researchgate.net/publication/358352335_Home_Automation_System_Using_ESP8266_Microcontroller_and_Blynk_Application

Likhitha U N, Manjunath R, and Darshan N (2024), IoT BASED HOME AUTOMATION SYSTEM WITH WI-FI AND ESP8266, International Journal of Creative Research Thoughts (IJCRT), vol. 12, issue 7, pp. c239-c243.

https://ijcrt.org/viewfulltext.php?&p_id=IJCRT2407267

APPENDIX

Conference Presentation Certificate



LOYOLA COLLEGE (AUTONOMOUS)

Nungambakkam, Chennai - 600034

Affiliated to the University of Madras & Re-accredited with 'A++' Grade by NAAC

DEPARTMENT OF COMPUTER SCIENCE AND APPLICATIONS, LCC

In Collaboration With

TAMIL NADU STATE COUNCIL FOR SCIENCE & TECHNOLOGY

TNSCST

CHRIST COLLEGE OF ARTS & SCIENCE, KILACHERY (CCASK)

LC/CS/NIAC2026/PP22

Date: 13/02/2026

CERTIFICATE OF APPRECIATION

This certificate is presented to Mr. /Ms. /Dr. /Prof. SAMUVEL JEBA KUMAR D

For presenting a paper in **National Conference on Innovations and Advancements in Computational Sciences** held on 12 - 02- 2026 and 13 - 02- 2026 at Loyola College.

Title: REAL-TIME BIRD SOUND PREDICTION USING DEEP LEARNING AND IOT

NIAC2026

Rev. Dr. A. Louis Arockiaraj SJ, Principal

Prof. L. Joseph Arockiasamy, Deputy Principal

Dr. J. Jerald Inico, Head of the Department

Dr. I. Justin Sophia, Coordinator

Dr. M. D. Anandaraj, Convenor

Source code

TensorFlow Bird Sound Training

```
import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, InputLayer, Dropout, Conv1D, Conv2D, Flatten, Reshape, MaxPooling1D, MaxPooling2D, AveragePooling2D, BatchNormalization, Permute, ReLU, Softmax, Activation

from tensorflow.keras.optimizers.legacy import Adam


EPOCHS = args.epochs or 50

LEARNING_RATE = args.learning_rate or 0.005

# If True, non-deterministic functions (e.g. shuffling batches) are not used.

# This is False by default.

ENSURE_DETERMINISM = args.ensure_determinism

# this controls the batch size, or you can manipulate the tf.data.Dataset objects yourself

BATCH_SIZE = args.batch_size or 32

if not ENSURE_DETERMINISM:

    train_dataset = train_dataset.shuffle(buffer_size=BATCH_SIZE*4)

train_dataset=train_dataset.batch(BATCH_SIZE, drop_remainder=False)

validation_dataset = validation_dataset.batch(BATCH_SIZE, drop_remainder=False)


# model architecture

model = Sequential()

model.add(Reshape((int(input_length / 40), 40), input_shape=(input_length, )))

model.add(Conv1D(8, kernel_size=3, padding='same', activation='relu'))

model.add(MaxPooling1D(pool_size=2, strides=2, padding='same'))

model.add(Dropout(0.25))

model.add(Conv1D(16, kernel_size=3, padding='same', activation='relu'))

model.add(MaxPooling1D(pool_size=2, strides=2, padding='same'))

model.add(Dropout(0.25))

model.add(Flatten())
```

```
model.add(Dense(classes, name='y_pred', activation='softmax'))


# this controls the learning rate
opt = Adam(learning_rate=LEARNING_RATE, beta_1=0.9, beta_2=0.999)
callbacks.append(BatchLoggerCallback(BATCH_SIZE, train_sample_count,
epochs=EPOCHS, ensure_determinism=ENSURE_DETERMINISM))


# train the neural network
model.compile(loss='categorical_crossentropy', optimizer=opt, metrics=["accuracy"])
model.fit(train_dataset, epochs=EPOCHS, validation_data=validation_dataset, verbose=2,
callbacks=callbacks)


# Use this flag to disable per-channel quantization for a model.
# This can reduce RAM usage for convolutional models, but may have
# an impact on accuracy.
disable_per_channel_quantization = False
```

Arduino BLE Acoustic Classification

```
#define EIDSP_QUANTIZE_FILTERBANK   0
#define EI_CLASSIFIER_SLICES_PER_MODEL_WINDOW 4
#define CONFIDENCE_THRESHOLD 0.70
#include <PDM.h>
#include <sam2003-project-1_inferencing.h>


typedef struct {
    signed short *buffers[2];
    unsigned char buf_select;
    unsigned char buf_ready;
    unsigned int buf_count;
    unsigned int n_samples;
} inference_t;
```

```
static inference_t inference;
static bool record_ready = false;
static signed short *sampleBuffer;
static bool debug_nn = false;
static int print_results = -(EI_CLASSIFIER_SLICES_PER_MODEL_WINDOW);

void setup()
{
    Serial.begin(115200);
    while (!Serial);

    // TX (Pin1/P1.03) --> ESP8266 RX (GPIO4)
    // RX (Pin0/P1.10) <-- ESP8266 TX (GPIO3)
    Serial1.begin(9600);

    Serial.println("============================");
    Serial.println("   Bird Sound Classifier");
    Serial.println("   Waiting for bird sound...");
    Serial.println("============================");

    run_classifier_init();
    if (microphone_inference_start(EI_CLASSIFIER_SLICE_SIZE) == false) {
        ei_printf("ERR: Could not allocate audio buffer (size %d)\r\n",
EI_CLASSIFIER_RAW_SAMPLE_COUNT);
        return;
    }
}

void loop()
{
```

```
bool m = microphone_inference_record();
if (!m) {
    ei_printf("ERR: Failed to record audio...\n");
    return;
}

signal_t signal;
signal.total_length = EI_CLASSIFIER_SLICE_SIZE;
signal.get_data = &microphone_audio_signal_get_data;
ei_impulse_result_t result = {0};

EI_IMPULSE_ERROR r = run_classifier_continuous(&signal, &result, debug_nn);
if (r != EI_IMPULSE_OK) {
    ei_printf("ERR: Failed to run classifier (%d)\n", r);
    return;
}

if (++print_results >= (EI_CLASSIFIER_SLICES_PER_MODEL_WINDOW)) {
    int best_index = 0;
    float best_value = 0.0;

    for (size_t ix = 0; ix < EI_CLASSIFIER_LABEL_COUNT; ix++) {
        if (result.classification[ix].value > best_value) {
            best_value = result.classification[ix].value;
            best_index = ix;
        }
    }

    if (best_value >= CONFIDENCE_THRESHOLD) {
        ei_printf("Predicted Bird: %s\n", result.classification[best_index].label);
```

```
            Serial1.println(result.classification[best_index].label);
        }
        print_results = 0;
    }
}

static void pdm_data_ready_inference_callback(void)
{
    int bytesAvailable = PDM.available();
    int bytesRead = PDM.read((char *)&sampleBuffer[0], bytesAvailable);

    if (record_ready == true) {
        for (int i = 0; i < bytesRead >> 1; i++) {
            inference.buffers[inference.buf_select][inference.buf_count++] = sampleBuffer[i];

            if (inference.buf_count >= inference.n_samples) {
                inference.buf_select ^= 1;
                inference.buf_count = 0;
                inference.buf_ready = 1;
            }
        }
    }
}

static bool microphone_inference_start(uint32_t n_samples)
{
    inference.buffers[0] = (signed short *)malloc(n_samples * sizeof(signed short));
    if (inference.buffers[0] == NULL) { return false; }

    inference.buffers[1] = (signed short *)malloc(n_samples * sizeof(signed short));
```

```
    if (inference.buffers[1] == NULL) {
        free(inference.buffers[0]);
        return false;
    }

    sampleBuffer = (signed short *)malloc((n_samples >> 1) * sizeof(signed short));
    if (sampleBuffer == NULL) {
        free(inference.buffers[0]);
        free(inference.buffers[1]);
        return false;
    }

    inference.buf_select = 0;
    inference.buf_count  = 0;
    inference.n_samples  = n_samples;
    inference.buf_ready  = 0;

    PDM.onReceive(&pdm_data_ready_inference_callback);
    PDM.setBufferSize((n_samples >> 1) * sizeof(int16_t));

    if (!PDM.begin(1, EI_CLASSIFIER_FREQUENCY)) {
        ei_printf("Failed to start PDM!");
    }

    PDM.setGain(127);
    record_ready = true;
    return true;
}

static bool microphone_inference_record(void)
```

```
{
    bool ret = true;
    if (inference.buf_ready == 1) {
        ret = false;
    }
    while (inference.buf_ready == 0) { delay(1); }
    inference.buf_ready = 0;
    return ret;
}

static int microphone_audio_signal_get_data(size_t offset, size_t length, float *out_ptr)
{
    numpy::int16_to_float(&inference.buffers[inference.buf_select ^ 1][offset], out_ptr,
length);
    return 0;
}
```

Node MCU ESP8266 Supabase Upload Code

```
/*
 * ESP8266 NodeMCU - Bird Receiver to Supabase
 * Receives bird names via SoftwareSerial and uploads to Supabase DB.
 */
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClientSecureBearSSL.h>
#include <SoftwareSerial.h>
#include <time.h>

// Communication Pins
#define RX_PIN 13  // D7
#define TX_PIN 15  // D8
```

```
SoftwareSerial ArduinoSerial(RX_PIN, TX_PIN);
// WiFi & Database Credentials
const char* ssid        = "narzo 50 5G";
const char* password    = "11111111";
const char* supabaseUrl = "https://sqifwksbcybzjkisdkxy.supabase.co/rest/v1/prediction";
const char* apiKey      =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InNxaW
Z3a3NiY3liempraXNka3h5Iiwicm9sZSI6ImFub24iLCJpYXQiOjE3NzQwMTE3ODUsImV
4cCI6MjA4OTU4Nzc4NX0.XRV7aGaVulaHM6vBRTITPOSnie5jBD3WFdQyqi4vw88";
const char* location    = " MCC,Chennai, Tamil Nadu";
void setup() {
  Serial.begin(115200);
  ArduinoSerial.begin(9600);

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  // Sync time for India (IST = UTC+5:30)
  configTime(19800, 0, "pool.ntp.org");
  delay(2000);
}

void loop() {
  if (ArduinoSerial.available() > 0) {
    String incomingBird = ArduinoSerial.readStringUntil('\n');
    incomingBird.trim();

    if (incomingBird.length() > 0) {
```

```
      Serial.println("\n--- Bird Detected ---");
      Serial.println("Species: " + incomingBird);

      delay(2000); // Visual hold
      sendToSupabase(incomingBird);
    }
  }
}

String getTime() {
  time_t now = time(nullptr);
  struct tm* t = localtime(&now);
  char buf[20];
  sprintf(buf, "%02d:%02d:%02d", t->tm_hour, t->tm_min, t->tm_sec);
  return String(buf);
}

String getDate() {
  time_t now = time(nullptr);
  struct tm* t = localtime(&now);
  char buf[20];
  sprintf(buf, "%04d-%02d-%02d", t->tm_year + 1900, t->tm_mon + 1, t->tm_mday);
  return String(buf);
}

String getTimestamp() {
  return getDate() + "T" + getTime() + "+05:30";
}

void sendToSupabase(String birdName) {
```

```
  if (WiFi.status() != WL_CONNECTED) return;

  BearSSL::WiFiClientSecure client;
  client.setInsecure();

  HTTPClient http;
  http.begin(client, supabaseUrl);
  http.addHeader("Content-Type", "application/json");
  http.addHeader("apikey", apiKey);
  http.addHeader("Authorization", "Bearer " + String(apiKey));

  String payload = "{";
  payload += "\"Bird_name\":\"" + birdName + "\",";
  payload += "\"Detected-at\":\"" + getTimestamp() + "\",";
  payload += "\"Data\":\"" + getDate() + "\",";
  payload += "\"location\":\"" + String(location) + "\"";
  payload += "}";
  int httpCode = http.POST(payload);
  if (httpCode == 201) {
    Serial.println("Data Synced to Supabase ");
  } else {
    Serial.println("Sync Error: " + String(httpCode));
  }
  http.end();
}
```